LABORATORY RECORD

# Computer Networks

15 experiments • C programming • Cisco Packet Tracer

Aim, theory, algorithm / procedure, diagrams, code, output, result and conclusion for every experiment

Name _____

Register No. _____

Course / Slot _____

Faculty _____

Semester _____

*Diagrams in this record are drawn schematically; add your own screenshots (with ID card) in the boxes provided.*

# Index


| No. | Experiment | Tool | Date | Sign |
|---|---|---|---|---|
| 1 | Error detection using even and odd parity check | C | | |
| 2 | Error detection using checksum | C | | |
| 3 | Error detection using Cyclic Redundancy Check | C | | |
| 4 | Error detection and correction using Hamming code | C | | |
| 5 | Flow control using stop-and-wait protocol | C | | |
| 6 | Flow control using go-back-N protocol (corrupted frame, lost frame and lost ACK) | C | | |
| 7 | Flow control using selective repeat ARQ | C | | |
| 8 | Implement IP addressing and default IP gateway using CPT | CPT | | |
| 9 | Implement RIP and OSPF using CPT | CPT | | |
| 10 | Implement the communication using SMTP and DNS using CPT | CPT | | |
| 11 | Implement unicast routing with the help of LSR using CPT | CPT | | |
| 12 | Implement unicast routing with the help of DVR using CPT | CPT | | |
| 13 | Implement unicast routing with the help of BGP using CPT | CPT | | |
| 14 | To perform traffic control operations by using TCP and UDP with the help of CPT | CPT | | |
| 15 | Develop a DNS client server to resolve the given host name (or) IP addresses | C + CPT | | |

# Before you start

Common set-up used by every experiment

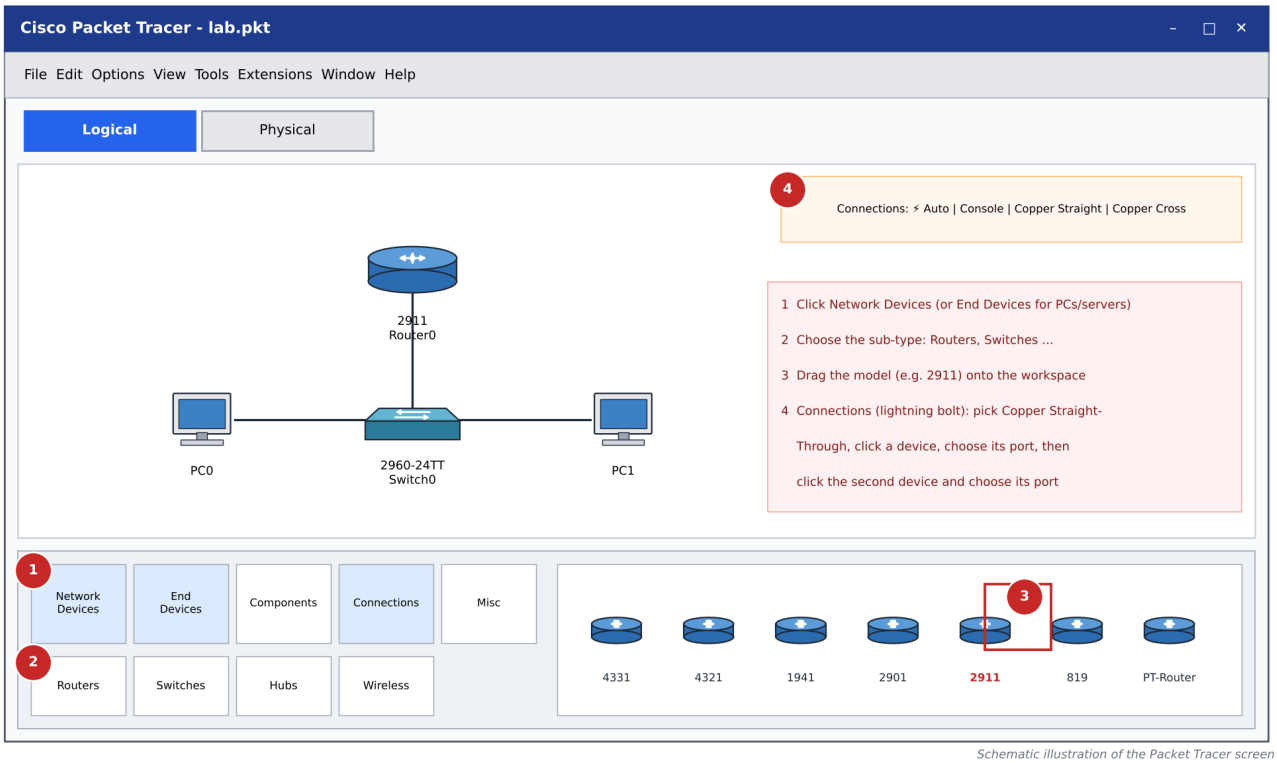
## Running the C programs (Experiments 1–7 and 15)

1. Open **Code::Blocks** → *File* → *New* → *Empty File* (or *New* → *Project* → *Console application* → *C*).
2. Paste the program and save it with the **.c** extension shown in the experiment (saving as .cpp also works but .c is correct for C).
3. Press **F9** (*Build* → *Build and run*). Type the inputs in the console window that opens.
4. On Linux / macOS or with MinGW on the command line: `gcc file.c -o file` then `./file`.
5. Take the output screenshot with your ID card visible, as in your earlier submissions.

**Tested:** Every program in this record was compiled with gcc and run; the output blocks are the actual program output for the inputs stated. The ARQ programs (Experiments 5–7) are deterministic simulations: you choose which frame or ACK is lost, so you can reproduce any case on demand.

## Packet Tracer basics (Experiments 8–15)

The figures below are labelled schematic drawings of the Packet Tracer screens, with red numbers showing where to click. Your Packet Tracer version may look slightly different, but the tabs and menu names are the same.




*Schematic illustration of the Packet Tracer screen*

*Figure B.1 – Placing devices and connecting cables*

1. **Add a device:** bottom-left device-type box → category (Network Devices / End Devices) → sub-type (Routers, Switches, PCs, Servers) → drag the model onto the workspace.
2. **Connect:** Connections (lightning bolt) → **Copper Straight-Through** for PC–switch, switch–router and router–router Gigabit links; **Copper Cross-Over** for PC–router or PC–PC; click device → choose port → click other device → choose port. (The automatic "⚡" cable also works but choosing ports yourself keeps them matching the procedure.)
3. **Link lights:** green = up, amber = switch port still starting (wait ~30 s), red = down (usually a router interface still in shutdown).

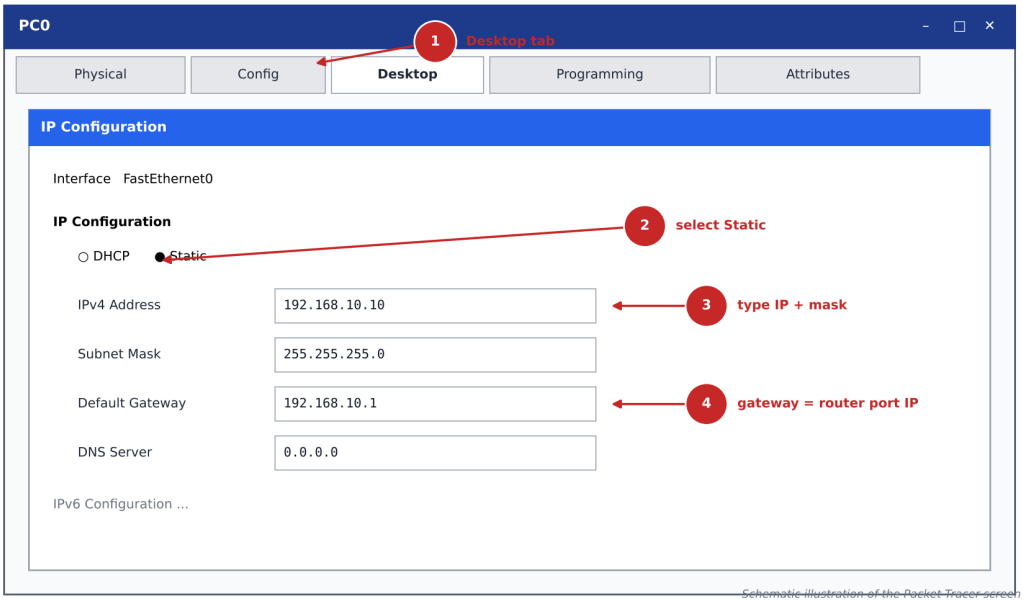



*Figure B.2 – PC → Desktop → IP Configuration (static)*

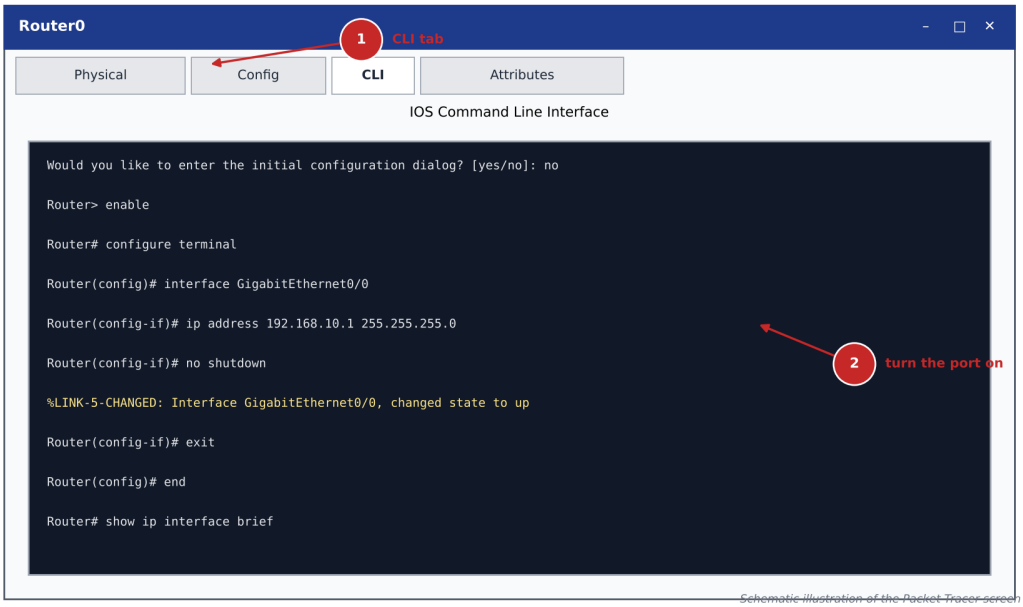



*Figure B.3 – Router → CLI tab: entering IOS commands*

## IOS CLI essentials

| Prompt | Mode | How to get there / leave |
|---|---|---|
| `Router>` | User EXEC | start here; enable → privileged |
| `Router#` | Privileged EXEC (show, ping, debug, write) | enable; disable to go back |
| `Router(config)#` | Global configuration | configure terminal; end or Ctrl+Z to leave |
| `Router(config-if)#` | Interface configuration | interface g0/0; exit |
| `Router(config-router)#` | Routing protocol configuration | router rip / ospf 1 / bgp 100; exit |

- Answer **no** to “Would you like to enter the initial configuration dialog?”.
- Commands can be abbreviated (`conf t`, `int g0/0`, `no shut`, `sh ip ro`); press Tab to complete and **?** for help.
- Save with `write memory` (or `copy running-config startup-config`) and save the .pkt file with *File → Save*.

- CLI outputs shown in this record are typical outputs: timers, ages, counters and port numbers in your own run will be slightly different.

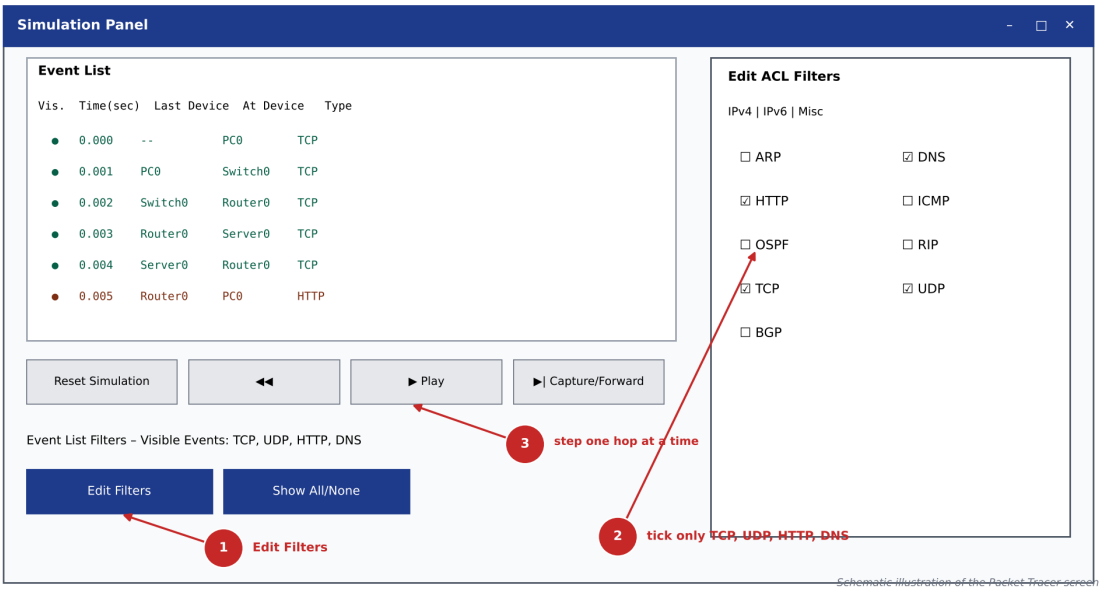



*Figure B.4 – Simulation mode: event list and filters*

1. Click **Simulation** (bottom-right, next to Realtime).
2. **Edit Filters** → choose only the protocols you want to watch (e.g. ICMP, TCP, UDP, DNS, OSPF).
3. Create traffic (ping, web browser, Add Simple PDU), then press **Capture/Forward** to move one hop at a time, or Play to run continuously.
4. Click an envelope (in the workspace or the event list) to open **PDU Information**: the OSI Model tab explains the device's decision; Inbound/Outbound PDU Details show every header field.

EXPERIMENT 01

# Error Detection using Even and Odd Parity Check

*Language: C*

Date: _____ Name: _____ Reg. No: _____

## Aim

To write a C program that generates an even or odd parity bit for a given binary data word at the sender, and checks the received codeword at the receiver to detect single-bit (odd-number-of-bit) transmission errors.

## Theory

A **parity bit** is one redundant bit appended to the data so that the total number of 1s in the codeword follows a rule. In **even parity** the total count of 1s (data + parity) must be even; in **odd parity** it must be odd. The receiver counts the 1s again: if the rule is broken, at least one bit has flipped.

Simple parity detects every error that changes an *odd* number of bits. It cannot detect an even number of flipped bits (two flips cancel out), and it cannot tell *which* bit is wrong, so it cannot correct errors.

| Data | No. of 1s | Even parity bit | Even codeword | Odd parity bit | Odd codeword |
|---|---|---|---|---|---|
| 1011001 | 4 | 0 | 10110010 | 1 | 10110011 |
| 1101 | 3 | 1 | 11011 | 0 | 11010 |
| 1111111 | 7 | 1 | 11111111 | 0 | 11111110 |

## Algorithm

1. Start.
2. Read the binary data string and validate that it contains only 0s and 1s.
3. Read the parity type (1 = even, 2 = odd).
4. Count the number of 1s in the data.
5. Even parity: parity = 0 if the count is even, else 1. Odd parity: parity = 1 if the count is even, else 0.
6. Append the parity bit to the data and display the transmitted codeword.
7. Read the received codeword and count its 1s.
8. If the count is even (for even parity) or odd (for odd parity) display “No error” and the data; otherwise display “Error detected”.
9. Stop.

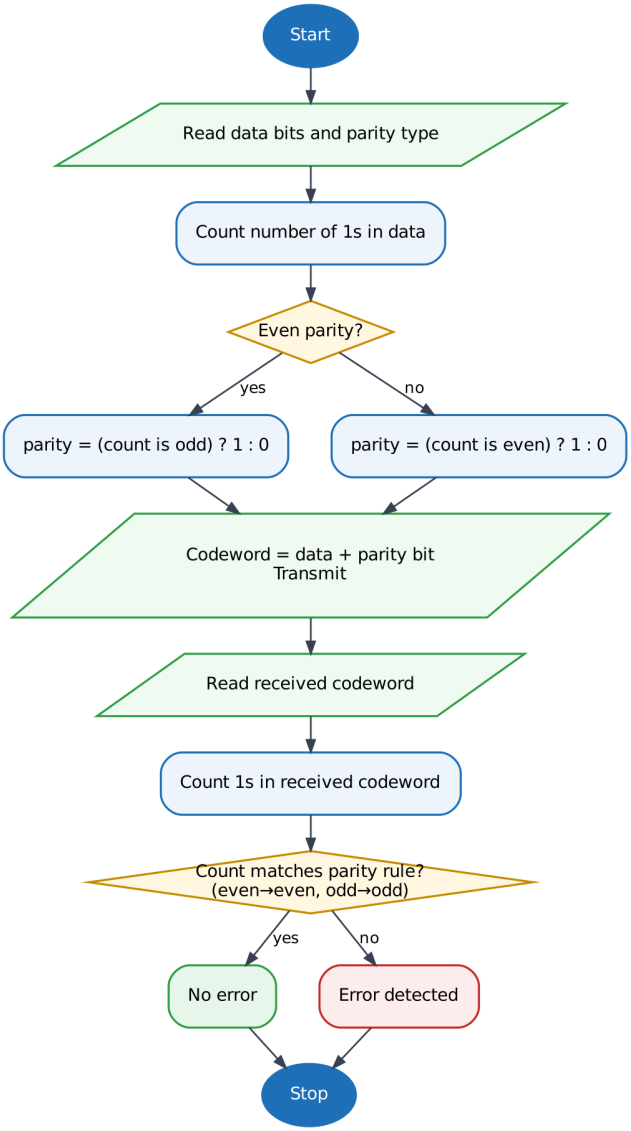



*Figure 1.1 – Flowchart of the parity check program*

## Program

```
/* Experiment 1: Error detection using even and odd parity check */
#include <stdio.h>
#include <string.h>

int count_ones(const char *s)
{
    int c = 0;
    for (int i = 0; s[i] != '\0'; i++)
        if (s[i] == '1')
            c++;
    return c;
}

int valid_binary(const char *s)
{
    for (int i = 0; s[i] != '\0'; i++)
        if (s[i] != '0' && s[i] != '1')
            return 0;
    return strlen(s) > 0;
}

int main(void)
{
    char data[100], codeword[102], received[102];
    int choice, ones, parity;

    printf("===== PARITY CHECK (SENDER SIDE) =====\n");
    printf("Enter binary data : ");
    scanf("%99s", data);
    if (!valid_binary(data)) {
```

```
        printf("Invalid input. Enter only 0s and 1s.\n");
        return 1;
    }

    printf("Choose parity type (1 = Even, 2 = Odd) : ");
    scanf("%d", &choice);

    ones = count_ones(data);
    if (choice == 1)               /* even: total number of 1s must be even */
        parity = (ones % 2 == 0) ? 0 : 1;
    else                           /* odd: total number of 1s must be odd  */
        parity = (ones % 2 == 0) ? 1 : 0;

    sprintf(codeword, "%s%d", data, parity);
    printf("Number of 1s in data : %d\n", ones);
    printf("Parity bit           : %d\n", parity);
    printf("Transmitted codeword : %s\n", codeword);

    printf("\n===== PARITY CHECK (RECEIVER SIDE) =====\n");
    printf("Enter received codeword : ");
    scanf("%101s", received);
    if (!valid_binary(received) || strlen(received) != strlen(codeword)) {
        printf("Invalid received codeword length/format.\n");
        return 1;
    }

    ones = count_ones(received);
    printf("Number of 1s in received codeword : %d\n", ones);

    if ((choice == 1 && ones % 2 == 0) || (choice == 2 && ones % 2 == 1))
        printf("Result : NO ERROR detected. Data = %.*s\n",
               (int)strlen(received) - 1, received);
    else
        printf("Result : ERROR DETECTED in received data!\n");

    return 0;
}
```

### How to run

- Code::Blocks: *File → New → Empty File*, paste the program, save it as **parity.c**, then press **F9** (Build and Run).
- Terminal: `gcc parity.c -o parity` then `./parity` (on Windows: `parity.exe`).
- Inputs used below: data **1011001**, even parity; then the same codeword with one bit flipped; then data 1101 with odd parity.

## Output

### Case 1 – Even parity, no error

```
$ ./parity
===== PARITY CHECK (SENDER SIDE) =====
Enter binary data : 1011001
Choose parity type (1 = Even, 2 = Odd) : 1
Number of 1s in data : 4
Parity bit           : 0
Transmitted codeword : 10110010

===== PARITY CHECK (RECEIVER SIDE) =====
Enter received codeword : 10110010
Number of 1s in received codeword : 4
Result : NO ERROR detected. Data = 1011001
```

### Case 2 – Even parity, bit 4 flipped during transmission

```
$ ./parity
===== PARITY CHECK (SENDER SIDE) =====
Enter binary data : 1011001
Choose parity type (1 = Even, 2 = Odd) : 1
Number of 1s in data : 4
Parity bit           : 0
Transmitted codeword : 10110010

===== PARITY CHECK (RECEIVER SIDE) =====
Enter received codeword : 10111010
Number of 1s in received codeword : 5
```

```
Result : ERROR DETECTED in received data!
```

### Case 3 – Odd parity, no error

```
$ ./parity
===== PARITY CHECK (SENDER SIDE) =====
Enter binary data : 1101
Choose parity type (1 = Even, 2 = Odd) : 2
Number of 1s in data : 3
Parity bit           : 0
Transmitted codeword : 11010

===== PARITY CHECK (RECEIVER SIDE) =====
Enter received codeword : 11010
Number of 1s in received codeword : 3
Result : NO ERROR detected. Data = 1101
```

*Output screenshot – Experiment 1*

(paste your own screenshot here, with your ID card visible)

## Result

The C program generated the correct even and odd parity bits (data 1011001 → codeword 10110010 with even parity) and the receiver detected the error when one bit was flipped (five 1s received instead of four).

## Conclusion

Parity check is the simplest error-detection technique: it costs only one extra bit and detects all odd-numbered bit errors, but it misses even-numbered errors and cannot locate or correct the faulty bit.

EXPERIMENT 02

# Error Detection using Checksum

*Language: C*

Date: _____ Name: _____ Reg. No: _____

## Aim

To write a C program that computes the one's-complement checksum of a set of data segments at the sender and verifies the segments and checksum at the receiver to detect transmission errors.

## Theory

The data is divided into *k* segments of *n* bits each. The sender adds all segments using **one's-complement arithmetic**: any carry out of the leftmost bit is wrapped around and added back to the rightmost bit. The **checksum** is the one's complement (bit inversion) of this sum and is sent along with the data.

The receiver adds all received segments *and* the checksum the same way. If nothing changed, the result is all 1s, so its complement is 0 → no error. Any non-zero complement means an error. The same idea (with 16-bit words) is used in the IP, TCP and UDP headers.

### Worked example (n = 8, k = 4)

| Step | Value | Comment |
|---|---|---|
| Segment 1 | `10011001` | |
| + Segment 2 | `11100010` | sum = 1 01111011 → wrap carry → 01111100 |
| + Segment 3 | `00100100` | sum = 10100000 |
| + Segment 4 | `10000100` | sum = 1 00100100 → wrap carry → 00100101 |
| Checksum | `11011010` | one's complement of 00100101 |
| Receiver sum | `11111111` | all four segments + checksum |
| Complement | `00000000` | zero → no error |

## Algorithm

1. Start.
2. Read the number of bits per segment n and the number of segments k.
3. sum = 0. For each segment: read it, sum = sum + segment; while the sum has more than n bits, add the carry back into the low n bits (wrap-around).
4. checksum = complement of sum (masked to n bits). Display the transmitted segments and checksum.
5. Receiver: read the k segments and the checksum, add all of them with wrap-around.
6. Complement the sum. If it is 0 display "No error", otherwise "Error detected".
7. Stop.

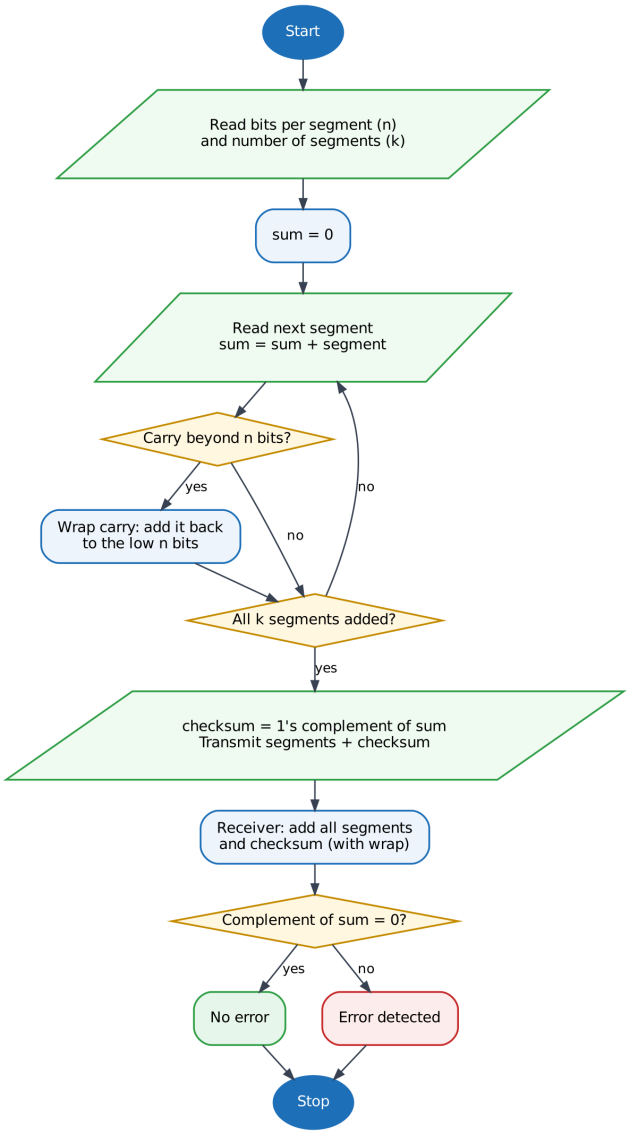



*Figure 2.1 – Flowchart of the checksum program*

## Program

```
/* Experiment 2: Error detection using Checksum (one's complement) */
#include <stdio.h>
#include <string.h>

int  bits;                      /* size of each segment in bits */

unsigned bin_to_int(const char *s)
{
    unsigned v = 0;
    for (int i = 0; s[i]; i++)
        v = (v << 1) | (s[i] - '0');
    return v;
}

void print_bin(unsigned v)
{
    for (int i = bits - 1; i >= 0; i--)
        putchar((v >> i) & 1 ? '1' : '0');
}

/* one's complement addition with end-around carry */
unsigned ones_add(unsigned a, unsigned b)
{
    unsigned mask = (1u << bits) - 1;
    unsigned sum = a + b;
    while (sum >> bits)                /* wrap the carry back in */
        sum = (sum & mask) + (sum >> bits);
    return sum;
}
```

```
int main(void)
{
    int k;
    char seg[33];
    unsigned data[20], sum = 0, checksum, mask;

    printf("===== CHECKSUM (SENDER SIDE) =====\n");
    printf("Enter number of bits per segment : ");
    scanf("%d", &bits);
    printf("Enter number of segments         : ");
    scanf("%d", &k);
    mask = (1u << bits) - 1;

    for (int i = 0; i < k; i++) {
        printf("Segment %d : ", i + 1);
        scanf("%32s", seg);
        data[i] = bin_to_int(seg);
        sum = ones_add(sum, data[i]);
        printf("    running sum = ");
        print_bin(sum);
        printf("\n");
    }
    checksum = (~sum) & mask;
    printf("Sum (wrapped)      : "); print_bin(sum);      printf("\n");
    printf("Checksum (1's comp): "); print_bin(checksum); printf("\n");
    printf("Transmitted        : ");
    for (int i = 0; i < k; i++) { print_bin(data[i]); printf(" "); }
    print_bin(checksum);
    printf("\n");

    printf("\n===== CHECKSUM (RECEIVER SIDE) =====\n");
    printf("Enter the %d received segments followed by the checksum:\n", k);
    sum = 0;
    for (int i = 0; i <= k; i++) {
        if (i < k) printf("Segment %d : ", i + 1);
        else       printf("Checksum  : ");
        scanf("%32s", seg);
        sum = ones_add(sum, bin_to_int(seg));
    }
    printf("Sum of all received : "); print_bin(sum);          printf("\n");
    printf("Complement          : "); print_bin((~sum) & mask); printf("\n");

    if (((~sum) & mask) == 0)
        printf("Result : Complement is 0 -> NO ERROR detected.\n");
    else
        printf("Result : Complement is non-zero -> ERROR DETECTED!\n");
    return 0;
}
```

### How to run

- Code::Blocks: *File* → *New* → *Empty File*, paste the program, save it as **checksum.c**, then press **F9** (Build and Run).
- Terminal: `gcc checksum.c -o checksum` then `./checksum` (on Windows: `checksum.exe`).
- Inputs used below: 8-bit segments 10011001, 11100010, 00100100, 10000100.

## Output

### Case 1 – No error

```
$ ./checksum
===== CHECKSUM (SENDER SIDE) =====
Enter number of bits per segment : 8
Enter number of segments         : 4
Segment 1 : 10011001
    running sum = 10011001
Segment 2 : 11100010
    running sum = 01111100
Segment 3 : 00100100
    running sum = 10100000
Segment 4 : 10000100
    running sum = 00100101
Sum (wrapped)      : 00100101
Checksum (1's comp): 11011010
Transmitted        : 10011001 11100010 00100100 10000100 11011010

===== CHECKSUM (RECEIVER SIDE) =====
Enter the 4 received segments followed by the checksum:
```

```
Segment 1 : 10011001
Segment 2 : 11100010
Segment 3 : 00100100
Segment 4 : 10000100
Checksum  : 11011010
Sum of all received : 11111111
Complement          : 00000000
Result : Complement is 0 -> NO ERROR detected.
```

### Case 2 – Segment 3 corrupted (00100100 received as 00101100)

```
$ ./checksum
===== CHECKSUM (SENDER SIDE) =====
Enter number of bits per segment : 8
Enter number of segments         : 4
Segment 1 : 10011001
    running sum = 10011001
Segment 2 : 11100010
    running sum = 01111100
Segment 3 : 00100100
    running sum = 10100000
Segment 4 : 10000100
    running sum = 00100101
Sum (wrapped)      : 00100101
Checksum (1's comp): 11011010
Transmitted        : 10011001 11100010 00100100 10000100 11011010

===== CHECKSUM (RECEIVER SIDE) =====
Enter the 4 received segments followed by the checksum:
Segment 1 : 10011001
Segment 2 : 11100010
Segment 3 : 00101100
Segment 4 : 10000100
Checksum  : 11011010
Sum of all received : 00001000
Complement          : 11110111
Result : Complement is non-zero -> ERROR DETECTED!
```

*Output screenshot – Experiment 2*

(paste your own screenshot here, with your ID card visible)

## Result

The program produced the checksum 11011010 for the four segments. When the data was received unchanged, the complement of the receiver’s sum was 00000000 (no error); when one segment was altered, the complement was 11110111 and the error was detected.

## Conclusion

Checksum detects most multi-bit errors with low overhead and simple addition, which is why it is used by IP, TCP and UDP. It is weaker than CRC because errors that cancel each other in the sum (for example the same bit flipped in two segments in opposite directions) go unnoticed.

EXPERIMENT
**03**

# Error Detection using Cyclic Redundancy Check (CRC)

*Language: C*

Date: _____ Name: _____ Reg. No: _____

## Aim

To write a C program that computes the CRC remainder of a data word using modulo-2 division by a generator polynomial, forms the transmitted codeword, and checks the received codeword for errors.

## Theory

In CRC the data bits are treated as the coefficients of a polynomial. Sender and receiver agree on a **generator polynomial** G(x) of degree r (r + 1 bits). The sender appends r zeros to the data, divides by G using **modulo-2 division** (subtraction is XOR, there are no carries), and replaces the zeros with the r-bit remainder. The codeword is then exactly divisible by G.

The receiver divides the received codeword by G. A remainder of all zeros means no error; any non-zero remainder means the frame was corrupted. CRC detects all single-bit errors, all double-bit errors, all odd-numbered errors (if G has the factor x + 1) and all burst errors of length ≤ r. It is used in Ethernet (CRC-32), HDLC and USB.

### Worked example: data 1101011011, generator 10011 ($x^4 + x + 1$)

```
Dividend (data + 4 zeros):  11010110110000
                            10011    <- XOR generator at bit 0
                            --------------
                            01001110110000
                             10011    <- XOR generator at bit 1
                            --------------
                            00000010110000
                                  10011    <- XOR generator at bit 6
                            --------------
                            00000000101000
                                    10011    <- XOR generator at bit 8
                            --------------
                            00000000001110

Remainder = last 4 bits = 1110
Transmitted codeword = 1101011011 | 1110 = 11010110111110
```

## Algorithm

1. Start.
2. Read the data bits and the generator bits; let r = (generator length − 1).
3. Append r zeros to the data to form the dividend.
4. For i = 0 to (dividend length − generator length): if bit i of the dividend is 1, XOR the generator into bits i … i + r.
5. The last r bits of the dividend are the CRC remainder. Codeword = data + remainder.
6. Receiver: read the received codeword and divide it by the generator in the same way.
7. If the remainder is all zeros display “No error”, otherwise “Error detected”.
8. Stop.

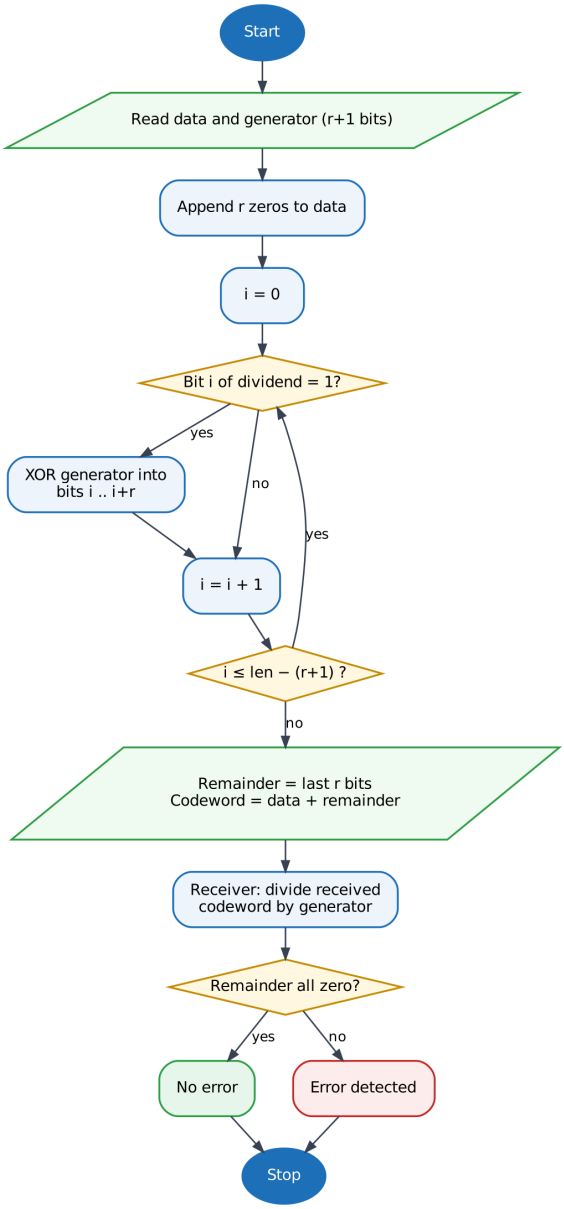



*Figure 3.1 – Flowchart of the CRC program*

## Program

```
/* Experiment 3: Error detection using Cyclic Redundancy Check (CRC) */
#include <stdio.h>
#include <string.h>

/* Modulo-2 division. 'dividend' is modified in place;
   the last (glen-1) bits hold the remainder at the end. */
void mod2_divide(char *dividend, const char *gen)
{
    int dlen = strlen(dividend), glen = strlen(gen);
    for (int i = 0; i <= dlen - glen; i++) {
        if (dividend[i] == '1')                    /* XOR only when leading bit is 1 */
            for (int j = 0; j < glen; j++)
                dividend[i + j] = (dividend[i + j] == gen[j]) ? '0' : '1';
    }
}

int main(void)
{
    char data[100], gen[30], temp[130], rem[30], codeword[130], recv[130];
    int dlen, glen;

    printf("===== CRC (SENDER SIDE) =====\n");
    printf("Enter data bits            : ");
    scanf("%99s", data);
    printf("Enter generator polynomial : ");
    scanf("%29s", gen);
```

```
    dlen = strlen(data);
    glen = strlen(gen);

    /* Step 1: append (glen-1) zeros */
    strcpy(temp, data);
    for (int i = 0; i < glen - 1; i++)
        temp[dlen + i] = '0';
    temp[dlen + glen - 1] = '\0';
    printf("Data after appending %d zeros : %s\n", glen - 1, temp);

    /* Step 2: divide */
    mod2_divide(temp, gen);
    strcpy(rem, temp + dlen);
    printf("CRC remainder              : %s\n", rem);

    /* Step 3: codeword = data + remainder */
    sprintf(codeword, "%s%s", data, rem);
    printf("Transmitted codeword       : %s\n", codeword);

    printf("\n===== CRC (RECEIVER SIDE) =====\n");
    printf("Enter received codeword    : ");
    scanf("%129s", recv);
    strcpy(temp, recv);
    mod2_divide(temp, gen);
    printf("Remainder at receiver      : %s\n", temp + strlen(recv) - (glen - 1));

    int error = 0;
    for (int i = strlen(recv) - (glen - 1); temp[i]; i++)
        if (temp[i] == '1')
            error = 1;
    if (error)
        printf("Result : Remainder is non-zero -> ERROR DETECTED!\n");
    else
        printf("Result : Remainder is zero -> NO ERROR. Data = %.*s\n",
               (int)strlen(recv) - (glen - 1), recv);
    return 0;
}
```

### How to run

- Code::Blocks: *File → New → Empty File*, paste the program, save it as **crc.c**, then press **F9** (Build and Run).
- Terminal: `gcc crc.c -o crc` then `./crc` (on Windows: `crc.exe`).
- Inputs used below: data 1101011011, generator 10011.

## Output

### Case 1 – Codeword received correctly

```
$ ./crc
===== CRC (SENDER SIDE) =====
Enter data bits            : 1101011011
Enter generator polynomial : 10011
Data after appending 4 zeros : 11010110110000
CRC remainder              : 1110
Transmitted codeword       : 11010110111110

===== CRC (RECEIVER SIDE) =====
Enter received codeword    : 11010110111110
Remainder at receiver      : 0000
Result : Remainder is zero -> NO ERROR. Data = 1101011011
```

### Case 2 – Bit 7 flipped in transit

```
$ ./crc
===== CRC (SENDER SIDE) =====
Enter data bits            : 1101011011
Enter generator polynomial : 10011
Data after appending 4 zeros : 11010110110000
CRC remainder              : 1110
Transmitted codeword       : 11010110111110

===== CRC (RECEIVER SIDE) =====
Enter received codeword    : 11010100111110
Remainder at receiver      : 1011
Result : Remainder is non-zero -> ERROR DETECTED!
```

*Output screenshot – Experiment 3*

(paste your own screenshot here, with your ID card visible)

## Result

For data 1101011011 and generator 10011 the program computed the CRC remainder 1110 and transmitted 11010110111110. The receiver obtained remainder 0000 for the correct codeword and a non-zero remainder (1011) for the corrupted one, so the error was detected.

## Conclusion

CRC uses polynomial (modulo-2) division to give much stronger error detection than parity or checksum for the same number of redundant bits, especially for burst errors, which is why it is used at the data-link layer.

EXPERIMENT
**04**

# Error Detection and Correction using Hamming Code

*Language: C*

Date: _____ Name: _____ Reg. No: _____

## Aim

To write a C program that encodes data bits into a Hamming code (even parity) at the sender, and at the receiver computes the syndrome to detect and correct a single-bit error.

## Theory

For m data bits, the number of parity (redundant) bits r is the smallest value satisfying $\mathbf{2^r \geq m + r + 1}$. For m = 4, r = 3 and the codeword has 7 bits: the classic Hamming(7,4) code. Bit positions are numbered from 1 (the rightmost bit). Parity bits sit at the positions that are powers of 2 (1, 2, 4, 8 …); data bits fill the remaining positions.

Parity bit $P_k$ (at position $k = 2^i$) covers every position whose binary index has bit i set, and is chosen so that the group has even parity. At the receiver, each check $C_k$ is recomputed; reading the checks as a binary number (C4 C2 C1) gives the **syndrome**, which is 0 if there is no error, or exactly the position of the flipped bit, which is then inverted to correct it.

| Position | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|---|
| Binary | 111 | 110 | 101 | 100 | 011 | 010 | 001 |
| Role | D | D | D | P4 | D | P2 | P1 |
| Data 1011 | 1 | 0 | 1 | ? | 1 | ? | ? |
| Codeword | 1 | 0 | 1 | 0 | 1 | 0 | 1 |

- P1 covers positions 1, 3, 5, 7 → bits 3, 5, 7 = 1, 1, 1 → P1 = 1
- P2 covers positions 2, 3, 6, 7 → bits 3, 6, 7 = 1, 0, 1 → P2 = 0
- P4 covers positions 4, 5, 6, 7 → bits 5, 6, 7 = 1, 0, 1 → P4 = 0
- Codeword (positions 7…1) = **1010101**. If position 5 flips (1000101), checks give C4 = 1, C2 = 0, C1 = 1 → syndrome $101_2 = 5$ → flip bit 5 back.

## Algorithm

1. Start.
2. Read the data bits (MSB first); m = number of data bits.
3. Find the smallest r such that $2^r \geq m + r + 1$; n = m + r.
4. Fill positions n down to 1: a power-of-2 position gets a 0 placeholder, every other position gets the next data bit.
5. For each parity position $p = 2^i$: XOR all positions $j \neq p$ with (j AND p) ≠ 0 and store the result at position p.
6. Display the Hamming codeword.
7. Receiver: read the codeword; for each p compute $C_p$ = XOR of all positions j with (j AND p) ≠ 0. syndrome = sum of p for which $C_p = 1$.
8. If syndrome = 0 there is no error; otherwise invert the bit at position = syndrome.
9. Extract and display the data bits (non-power-of-2 positions).
10. Stop.

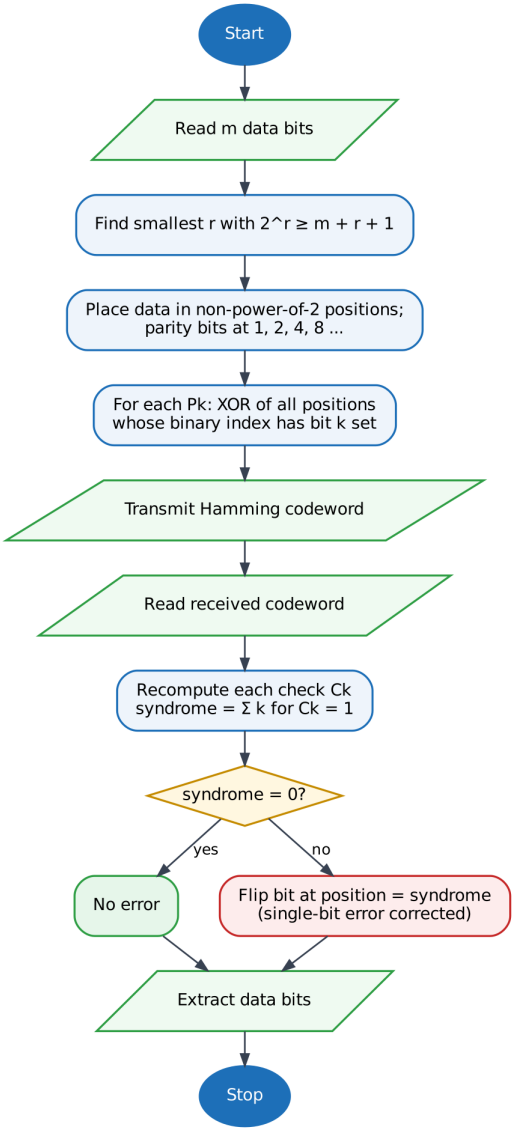



*Figure 4.1 – Flowchart of the Hamming code program*

## Program

```
/* Experiment 4: Error detection and correction using Hamming Code (even parity) */
#include <stdio.h>

int main(void)
{
    int m, r = 0, n, data[32], code[64], recv[64];
    char s[40];

    printf("===== HAMMING CODE (SENDER SIDE) =====\n");
    printf("Enter data bits (MSB first) : ");
    scanf("%39s", s);
    for (m = 0; s[m]; m++)
        data[m] = s[m] - '0';

    /* number of redundant bits: 2^r >= m + r + 1 */
    while ((1 << r) < m + r + 1)
        r++;
    n = m + r;
    printf("Data bits m = %d, Parity bits r = %d, Codeword length n = %d\n", m, r, n);

    /* Positions are numbered 1..n from the RIGHT (LSB side).
       Data bits fill non-power-of-2 positions, starting from position n downward
       with the MSB of the data. */
    int k = 0;
    for (int pos = n; pos >= 1; pos--) {
        if ((pos & (pos - 1)) == 0)
            code[pos] = 0;                  /* parity placeholder */
        else
            code[pos] = data[k++];
    }
```

```
    /* compute each parity bit P(2^i) */
    for (int i = 0; i < r; i++) {
        int p = 1 << i, parity = 0;
        for (int pos = 1; pos <= n; pos++)
            if ((pos & p) && pos != p)
                parity ^= code[pos];
        code[p] = parity;
        printf("P%-2d (covers positions with bit %d set) = %d\n", p, i, parity);
    }

    printf("Hamming codeword (pos %d..1)  : ", n);
    for (int pos = n; pos >= 1; pos--)
        printf("%d", code[pos]);
    printf("\n");

    printf("\n===== HAMMING CODE (RECEIVER SIDE) =====\n");
    printf("Enter received codeword (%d bits) : ", n);
    scanf("%39s", s);
    for (int pos = n, i = 0; pos >= 1; pos--, i++)
        recv[pos] = s[i] - '0';

    int syndrome = 0;
    for (int i = 0; i < r; i++) {
        int p = 1 << i, c = 0;
        for (int pos = 1; pos <= n; pos++)
            if (pos & p)
                c ^= recv[pos];
        printf("Check C%-2d = %d\n", p, c);
        if (c)
            syndrome |= p;
    }

    if (syndrome == 0) {
        printf("Syndrome = 0 -> NO ERROR in received codeword.\n");
    } else {
        printf("Syndrome = %d -> ERROR at bit position %d (from right)\n", syndrome, syndrome);
        if (syndrome <= n) {
            recv[syndrome] ^= 1;
            printf("Corrected codeword : ");
            for (int pos = n; pos >= 1; pos--)
                printf("%d", recv[pos]);
            printf("\n");
        }
    }

    printf("Extracted data     : ");
    for (int pos = n; pos >= 1; pos--)
        if ((pos & (pos - 1)) != 0)
            printf("%d", recv[pos]);
    printf("\n");
    return 0;
}
```

### How to run

- Code::Blocks: *File → New → Empty File*, paste the program, save it as **hamming.c**, then press **F9** (Build and Run).
- Terminal: `gcc hamming.c -o hamming` then `./hamming` (on Windows: `hamming.exe`).
- Inputs used below: data 1011; received codeword correct, then with bit 5 flipped.

## Output

### Case 1 – No error

```
$ ./hamming
===== HAMMING CODE (SENDER SIDE) =====
Enter data bits (MSB first) : 1011
Data bits m = 4, Parity bits r = 3, Codeword length n = 7
P1  (covers positions with bit 0 set) = 1
P2  (covers positions with bit 1 set) = 0
P4  (covers positions with bit 2 set) = 0
Hamming codeword (pos 7..1)  : 1010101

===== HAMMING CODE (RECEIVER SIDE) =====
Enter received codeword (7 bits) : 1010101
Check C1  = 0
Check C2  = 0
```

```
Check C4  = 0
Syndrome = 0 -> NO ERROR in received codeword.
Extracted data     : 1011
```

### Case 2 – Single-bit error at position 5

```
$ ./hamming
===== HAMMING CODE (SENDER SIDE) =====
Enter data bits (MSB first) : 1011
Data bits m = 4, Parity bits r = 3, Codeword length n = 7
P1  (covers positions with bit 0 set) = 1
P2  (covers positions with bit 1 set) = 0
P4  (covers positions with bit 2 set) = 0
Hamming codeword (pos 7..1)  : 1010101

===== HAMMING CODE (RECEIVER SIDE) =====
Enter received codeword (7 bits) : 1000101
Check C1  = 1
Check C2  = 0
Check C4  = 1
Syndrome = 5 -> ERROR at bit position 5 (from right)
Corrected codeword : 1010101
Extracted data     : 1011
```

*Output screenshot – Experiment 4*

(paste your own screenshot here, with your ID card visible)

## Result

Data 1011 was encoded as the Hamming codeword 1010101 (r = 3 parity bits). When bit 5 was flipped, the receiver computed syndrome 5, corrected the bit to recover 1010101, and extracted the original data 1011.

## Conclusion

Hamming code adds $\log_2$-order redundancy and can both detect and correct any single-bit error, because the syndrome directly gives the position of the faulty bit. It is used where retransmission is expensive, for example in ECC memory.

EXPERIMENT 05

# Flow Control using Stop-and-Wait Protocol

*Language: C*

Date: _____ Name: _____ Reg. No: _____

## Aim

To simulate the Stop-and-Wait ARQ protocol in C, showing how the sender waits for an acknowledgement after every frame and recovers from a lost frame and a lost acknowledgement using a timer, retransmission and alternating sequence numbers.

## Theory

In Stop-and-Wait ARQ the sender transmits **one frame**, starts a timer and waits. It sends the next frame only after the acknowledgement (ACK) arrives. Frames carry a 1-bit sequence number (0, 1, 0, 1 ...) and the ACK carries the number of the *next* frame expected.

- **Lost / corrupted frame:** no ACK comes back, the timer expires and the sender retransmits the same frame.
- **Lost ACK:** the receiver already has the frame, so the retransmitted copy carries a sequence number it has already seen; it discards the **duplicate** but sends the ACK again.
- Flow control is automatic: the sender can never be more than one frame ahead of the receiver.
- Efficiency is low on long links because the sender is idle for a full round-trip time per frame: U = 1 / (1 + 2a) where a = propagation time / transmission time.

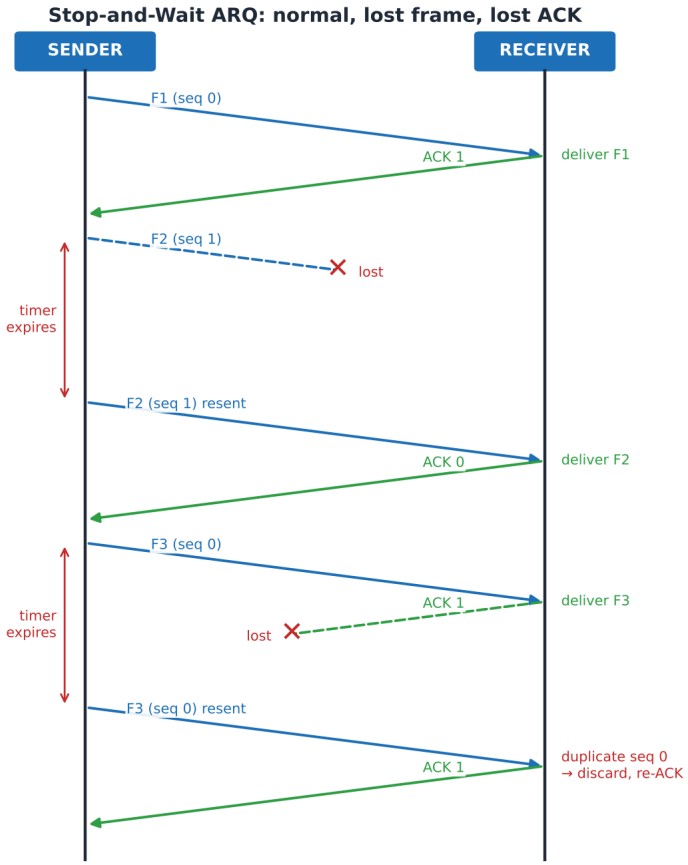



*Figure 5.1 – Stop-and-Wait timing: frame 2 lost, ACK of frame 3 lost*

## Algorithm

1. Start.

2. Read the total number of frames, the frame to lose and the frame whose ACK is lost.
3. Sender: seq = 0. Receiver: expected = 0.
4. Send frame f with sequence number seq and start the timer.
5. If the frame is lost, wait for the timeout and go back to step 4 (retransmit).
6. Receiver: if seq = expected, accept and deliver the frame and toggle expected; otherwise discard it as a duplicate. In both cases send ACK expected.
7. If the ACK is lost, wait for the timeout and go back to step 4.
8. On receiving the ACK, stop the timer, toggle seq and move to the next frame.
9. Repeat until all frames are delivered; print total transmissions and efficiency.
10. Stop.

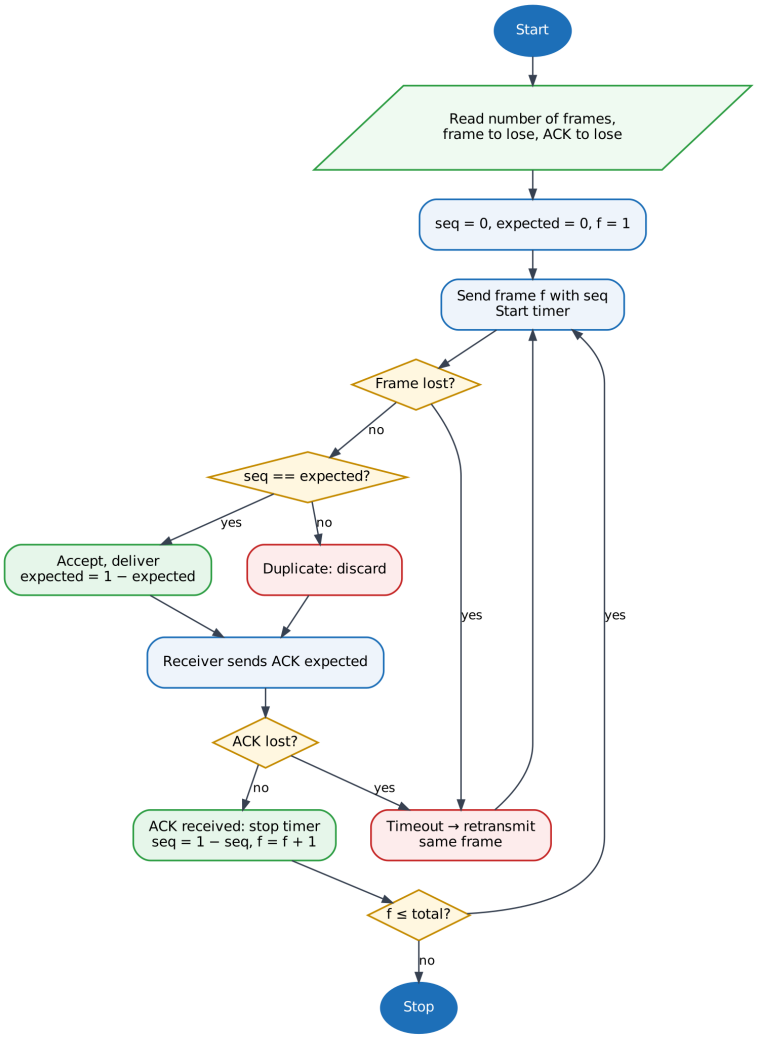



*Figure 5.2 – Flowchart of the Stop-and-Wait program*

## Program

```
/* Experiment 5: Flow control using Stop-and-Wait ARQ
   Sequence numbers alternate 0,1,0,1...  The user chooses which frame
   is lost and which ACK is lost so every case can be demonstrated. */
#include <stdio.h>

int main(void)
{
    int total, lost_frame, lost_ack;
    int seq = 0;            /* sender's current sequence number   */
    int expected = 0;       /* receiver's expected sequence number */
    int time = 0, transmissions = 0;
    const int TIMEOUT = 3;

    printf("===== STOP-AND-WAIT ARQ SIMULATION =====\n");
    printf("Enter total number of frames          : ");
    scanf("%d", &total);
    printf("Enter frame number to LOSE (0 = none) : ");
```

```
    scanf("%d", &lost_frame);
    printf("Enter frame whose ACK is LOST (0=none): ");
    scanf("%d", &lost_ack);
    printf("\n");

    for (int f = 1; f <= total; f++) {
        int delivered = 0;
        while (!delivered) {
            transmissions++;
            printf("t=%2d  [Sender]   Sends Frame %d (Seq %d), starts timer\n", time, f, seq);

            if (f == lost_frame) {                      /* frame lost on the channel */
                printf("t=%2d  [Channel]  Frame %d LOST\n", time + 1, f);
                time += TIMEOUT;
                printf("t=%2d  [Sender]   TIMEOUT! No ACK -> retransmit Frame %d\n", time, f);
                lost_frame = 0;                         /* lose it only once */
                continue;
            }

            /* frame reaches receiver */
            if (seq == expected) {
                printf("t=%2d  [Receiver] Frame %d (Seq %d) accepted, delivered to upper layer\n",
                       time + 1, f, seq);
                expected = 1 - expected;
            } else {
                printf("t=%2d  [Receiver] DUPLICATE Frame %d (Seq %d) -> discarded\n",
                       time + 1, f, seq);
            }
            printf("t=%2d  [Receiver] Sends ACK %d\n", time + 1, expected);

            if (f == lost_ack) {                        /* ACK lost on the channel */
                printf("t=%2d  [Channel]  ACK %d LOST\n", time + 2, expected);
                time += TIMEOUT;
                printf("t=%2d  [Sender]   TIMEOUT! No ACK -> retransmit Frame %d\n", time, f);
                lost_ack = 0;
                continue;
            }

            time += 2;
            printf("t=%2d  [Sender]   ACK %d received, timer stopped\n", time, expected);
            seq = 1 - seq;
            delivered = 1;
        }
        printf("------------------------------------------------------------\n");
    }
    printf("All %d frames delivered using %d transmissions.\n", total, transmissions);
    printf("Efficiency = %d/%d = %.1f%%\n", total, transmissions, 100.0 * total / transmissions);
    return 0;
}
```

### How to run

- Code::Blocks: *File* → *New* → *Empty File*, paste the program, save it as **stop_and_wait.c**, then press **F9** (Build and Run).
- Terminal: `gcc stop_and_wait.c -o stop_and_wait` then `./stop_and_wait` (on Windows: `stop_and_wait.exe`).
- Inputs used below: 4 frames, frame 2 lost, ACK of frame 3 lost.

## Output

```
$ ./stop_and_wait
===== STOP-AND-WAIT ARQ SIMULATION =====
Enter total number of frames          : 4
Enter frame number to LOSE (0 = none) : 2
Enter frame whose ACK is LOST (0=none): 3

t= 0  [Sender]   Sends Frame 1 (Seq 0), starts timer
t= 1  [Receiver] Frame 1 (Seq 0) accepted, delivered to upper layer
t= 1  [Receiver] Sends ACK 1
t= 2  [Sender]   ACK 1 received, timer stopped
------------------------------------------------------------
t= 2  [Sender]   Sends Frame 2 (Seq 1), starts timer
t= 3  [Channel]  Frame 2 LOST
t= 5  [Sender]   TIMEOUT! No ACK -> retransmit Frame 2
t= 5  [Sender]   Sends Frame 2 (Seq 1), starts timer
t= 6  [Receiver] Frame 2 (Seq 1) accepted, delivered to upper layer
t= 6  [Receiver] Sends ACK 0
```

```
t= 7  [Sender]   ACK 0 received, timer stopped
------------------------------------------------------------
t= 7  [Sender]   Sends Frame 3 (Seq 0), starts timer
t= 8  [Receiver] Frame 3 (Seq 0) accepted, delivered to upper layer
t= 8  [Receiver] Sends ACK 1
t= 9  [Channel]  ACK 1 LOST
t=10  [Sender]   TIMEOUT! No ACK -> retransmit Frame 3
t=10  [Sender]   Sends Frame 3 (Seq 0), starts timer
t=11  [Receiver] DUPLICATE Frame 3 (Seq 0) -> discarded
t=11  [Receiver] Sends ACK 1
t=12  [Sender]   ACK 1 received, timer stopped
------------------------------------------------------------
t=12  [Sender]   Sends Frame 4 (Seq 1), starts timer
t=13  [Receiver] Frame 4 (Seq 1) accepted, delivered to upper layer
t=13  [Receiver] Sends ACK 0
t=14  [Sender]   ACK 0 received, timer stopped
------------------------------------------------------------
All 4 frames delivered using 6 transmissions.
Efficiency = 4/6 = 66.7%
```

*Output screenshot – Experiment 5*

(paste your own screenshot here, with your ID card visible)

## Result

All 4 frames were delivered in order using 6 transmissions. The lost frame 2 was resent after the timeout, and when the ACK for frame 3 was lost, the retransmitted frame 3 was recognised as a duplicate (same sequence number 0), discarded and acknowledged again.

## Conclusion

Stop-and-Wait ARQ gives reliable, in-order delivery and simple flow control using only 1-bit sequence numbers, timers and retransmission, but it keeps the link idle while waiting for each ACK, so its throughput is poor on links with a large bandwidth-delay product.

EXPERIMENT
06

# Flow Control using Go-Back-N Protocol (Corrupted Frame, Lost Frame and Lost ACK)

*Language: C*

Date: _____ Name: _____ Reg. No: _____

## Aim

To simulate the Go-Back-N ARQ sliding-window protocol in C and observe how it handles three error cases: (a) a corrupted frame, (b) a lost frame and (c) a lost acknowledgement.

## Theory

Go-Back-N lets the sender transmit up to **N** frames (the window) without waiting for ACKs. With m-bit sequence numbers the window size can be at most $2^m - 1$. The receiver window is **1**: it accepts frames only in order and discards anything else. ACKs are **cumulative**: ACK k means all frames up to k − 1 have arrived.

- **Corrupted frame:** the receiver detects the error (checksum/CRC fails) and discards the frame and every frame after it (they are out of order). No ACK covers the bad frame, so its timer expires and the sender goes back and resends that frame and all following frames.
- **Lost frame:** identical recovery – the receiver sees the next frames out of order, discards them, and the sender times out and goes back N.
- **Lost ACK:** because ACKs are cumulative, a later ACK confirms the earlier frame too, so nothing needs resending. Only if no later ACK arrives (the lost ACK was the last in the window) does the timer expire; the receiver then discards the duplicate frame and re-sends the ACK.

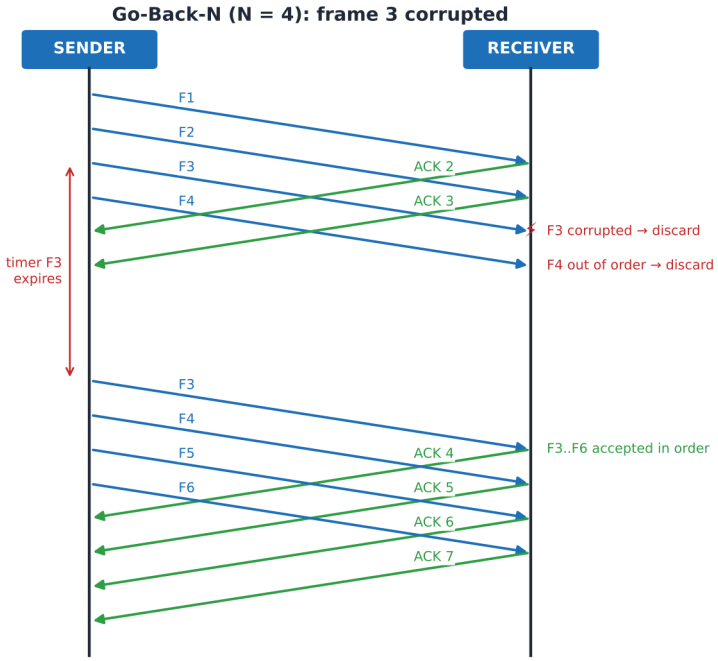



*Figure 6.1 – Go-Back-N, frame 3 corrupted: frames 3–6 are resent*

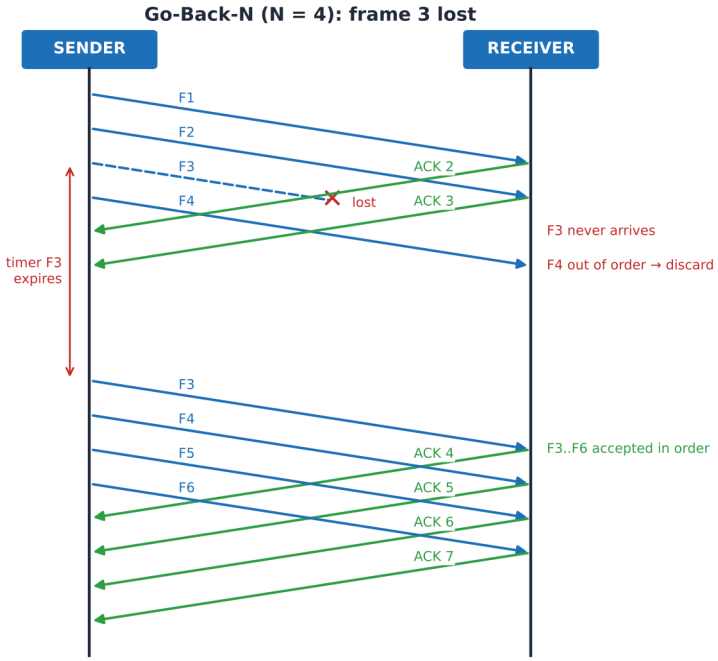



*Figure 6.2 – Go-Back-N, frame 3 lost*

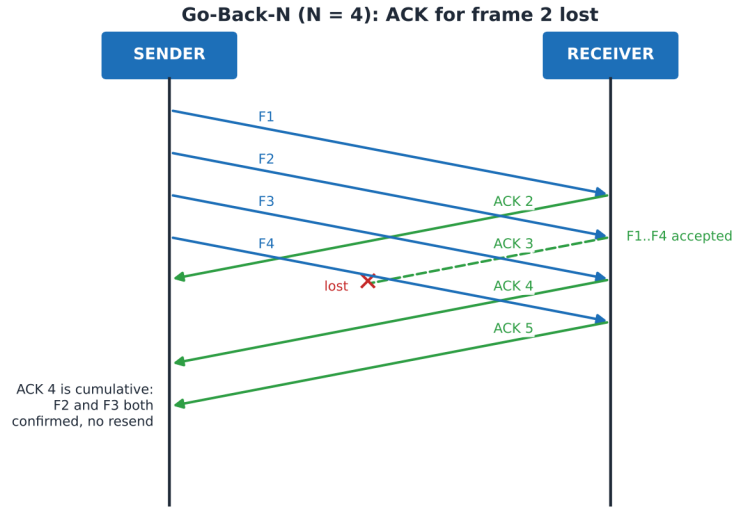



*Figure 6.3 – Go-Back-N, ACK of frame 2 lost: covered by cumulative ACK 4*

## Algorithm

1. Start.
2. Read the case (1 corrupted frame, 2 lost frame, 3 lost ACK), total frames, window size N and the frame number where the error occurs.
3. base = 1 (oldest unacknowledged frame), expected = 1 (receiver).
4. Send all frames from base to min(base + N − 1, total).
5. For each frame in order: if it is the corrupted/lost frame, discard it; else if frame ≠ expected, discard it as out-of-order (or duplicate → re-ACK); else accept it, expected = frame + 1 and send ACK expected (unless this ACK is the one lost).
6. The sender sets base to the highest ACK received (cumulative ACK).
7. If some frame in the window is still unacknowledged, its timer expires: go back to base and resend the whole window from there. Otherwise slide the window.
8. Repeat while base ≤ total; then display total transmissions and retransmissions.
9. Stop.

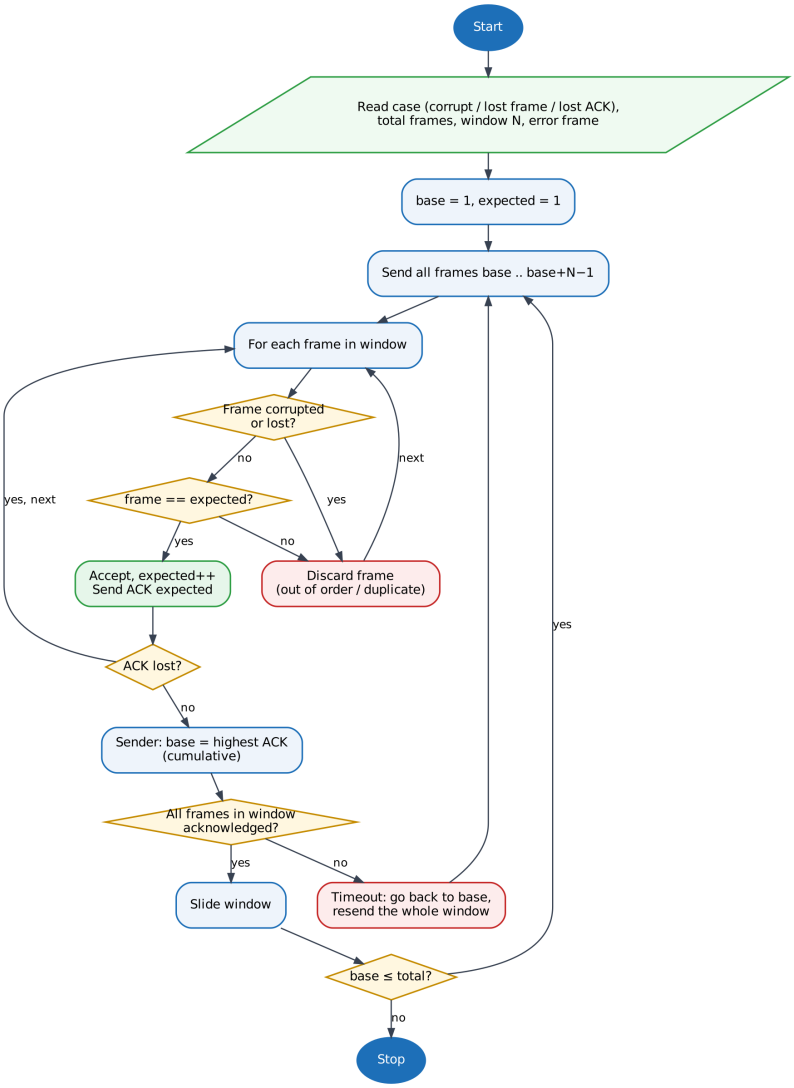



*Figure 6.4 – Flowchart of the Go-Back-N program*

## Program

```
/* Experiment 6: Flow control using Go-Back-N ARQ
   Case 1: corrupted frame   Case 2: lost frame   Case 3: lost ACK
   ACK k means "all frames up to k-1 received, next expected is k" (cumulative). */
#include <stdio.h>

int main(void)
{
    int total, N, choice, err;
    int base = 1, expected = 1, transmissions = 0, done = 0;
    const char *names[] = {"", "CORRUPTED FRAME", "LOST FRAME", "LOST ACK"};

    printf("===== GO-BACK-N ARQ SIMULATION =====\n");
    printf("1. Corrupted frame\n2. Lost frame\n3. Lost ACK\n");
    printf("Enter your choice            : ");
    scanf("%d", &choice);
    if (choice < 1 || choice > 3) { printf("Invalid choice\n"); return 1; }
    printf("Enter total number of frames : ");
    scanf("%d", &total);
    printf("Enter window size (N)        : ");
    scanf("%d", &N);
    printf("Enter the frame number for the error : ");
    scanf("%d", &err);
    printf("\n--- Case: %s at frame %d, window = %d ---\n", names[choice], err, N);

    while (base <= total) {
        int end = base + N - 1;
        if (end > total) end = total;
        int lastAck = -1, ack_missing = 0;

        printf("\n[Sender]   Window [%d..%d] -> sending frames:", base, end);
        for (int f = base; f <= end; f++) printf(" %d", f);
```

```
        printf("\n");
        transmissions += end - base + 1;

        for (int f = base; f <= end; f++) {
            if (!done && f == err && choice == 1) {
                printf("[Receiver] Frame %d CORRUPTED (checksum failed) -> discarded\n", f);
                done = 1;
                continue;
            }
            if (!done && f == err && choice == 2) {
                printf("[Channel]  Frame %d LOST in transit\n", f);
                done = 1;
                continue;
            }
            if (f > expected) {
                printf("[Receiver] Frame %d out of order (expecting %d) -> discarded\n", f, expected);
                continue;
            }
            if (f < expected) {
                printf("[Receiver] Frame %d is a DUPLICATE -> discarded, re-sends ACK %d\n", f,
                    expected);
                lastAck = expected;
                continue;
            }
            expected = f + 1;
            printf("[Receiver] Frame %d accepted, sends ACK %d\n", f, expected);

            if (!done && f == err && choice == 3) {
                printf("[Channel]  ACK %d LOST in transit\n", expected);
                done = 1;
                ack_missing = 1;
                continue;
            }
            if (ack_missing) {
                printf("[Sender]   ACK %d received (cumulative) -> frames up to %d confirmed,\n"
                       "           so the lost ACK does not matter\n", expected, f);
                ack_missing = 0;
            } else {
                printf("[Sender]   ACK %d received\n", expected);
            }
            lastAck = expected;
        }

        if (lastAck > base) base = lastAck;               /* slide window */
        if (base <= end) {
            printf("[Sender]   TIMEOUT for frame %d -> Go back to frame %d and resend window\n",
                   base, base);
        } else {
            printf("[Sender]   Window slides to base %d\n", base);
        }
        printf("-----------------------------------------------------------");
    }
    printf("\n\nAll %d frames delivered. Total transmissions = %d (retransmissions = %d)\n",
           total, transmissions, transmissions - total);
    return 0;
}
```

### How to run

- Code::Blocks: *File → New → Empty File*, paste the program, save it as **go_back_n.c**, then press **F9** (Build and Run).
- Terminal: `gcc go_back_n.c -o go_back_n` then `./go_back_n` (on Windows: `go_back_n.exe`).
- Inputs used below: 7 frames, window 4, error at frame 3 (cases 1 and 2), lost ACK of frame 2 (case 3a) and lost ACK at the end of a window (case 3b).

## Output

### Case 1 – Corrupted frame

```
$ ./go_back_n
===== GO-BACK-N ARQ SIMULATION =====
1. Corrupted frame
2. Lost frame
3. Lost ACK
Enter your choice            : 1
Enter total number of frames : 7
Enter window size (N)        : 4
```

```
Enter the frame number for the error : 3

--- Case: CORRUPTED FRAME at frame 3, window = 4 ---

[Sender]   Window [1..4] -> sending frames: 1 2 3 4
[Receiver] Frame 1 accepted, sends ACK 2
[Sender]   ACK 2 received
[Receiver] Frame 2 accepted, sends ACK 3
[Sender]   ACK 3 received
[Receiver] Frame 3 CORRUPTED (checksum failed) -> discarded
[Receiver] Frame 4 out of order (expecting 3) -> discarded
[Sender]   TIMEOUT for frame 3 -> Go back to frame 3 and resend window
------------------------------------------------------------
[Sender]   Window [3..6] -> sending frames: 3 4 5 6
[Receiver] Frame 3 accepted, sends ACK 4
[Sender]   ACK 4 received
[Receiver] Frame 4 accepted, sends ACK 5
[Sender]   ACK 5 received
[Receiver] Frame 5 accepted, sends ACK 6
[Sender]   ACK 6 received
[Receiver] Frame 6 accepted, sends ACK 7
[Sender]   ACK 7 received
[Sender]   Window slides to base 7
------------------------------------------------------------
[Sender]   Window [7..7] -> sending frames: 7
[Receiver] Frame 7 accepted, sends ACK 8
[Sender]   ACK 8 received
[Sender]   Window slides to base 8
------------------------------------------------------------

All 7 frames delivered. Total transmissions = 9 (retransmissions = 2)
```

## Case 2 – Lost frame

```
$ ./go_back_n
===== GO-BACK-N ARQ SIMULATION =====
1. Corrupted frame
2. Lost frame
3. Lost ACK
Enter your choice            : 2
Enter total number of frames : 7
Enter window size (N)        : 4
Enter the frame number for the error : 3

--- Case: LOST FRAME at frame 3, window = 4 ---

[Sender]   Window [1..4] -> sending frames: 1 2 3 4
[Receiver] Frame 1 accepted, sends ACK 2
[Sender]   ACK 2 received
[Receiver] Frame 2 accepted, sends ACK 3
[Sender]   ACK 3 received
[Channel]  Frame 3 LOST in transit
[Receiver] Frame 4 out of order (expecting 3) -> discarded
[Sender]   TIMEOUT for frame 3 -> Go back to frame 3 and resend window
------------------------------------------------------------
[Sender]   Window [3..6] -> sending frames: 3 4 5 6
[Receiver] Frame 3 accepted, sends ACK 4
[Sender]   ACK 4 received
[Receiver] Frame 4 accepted, sends ACK 5
[Sender]   ACK 5 received
[Receiver] Frame 5 accepted, sends ACK 6
[Sender]   ACK 6 received
[Receiver] Frame 6 accepted, sends ACK 7
[Sender]   ACK 7 received
[Sender]   Window slides to base 7
------------------------------------------------------------
[Sender]   Window [7..7] -> sending frames: 7
[Receiver] Frame 7 accepted, sends ACK 8
[Sender]   ACK 8 received
[Sender]   Window slides to base 8
------------------------------------------------------------

All 7 frames delivered. Total transmissions = 9 (retransmissions = 2)
```

## Case 3a – Lost ACK in the middle of the window

```
$ ./go_back_n
===== GO-BACK-N ARQ SIMULATION =====
1. Corrupted frame
```

```
2. Lost frame
3. Lost ACK
Enter your choice            : 3
Enter total number of frames : 7
Enter window size (N)        : 4
Enter the frame number for the error : 2

--- Case: LOST ACK at frame 2, window = 4 ---

[Sender]   Window [1..4] -> sending frames: 1 2 3 4
[Receiver] Frame 1 accepted, sends ACK 2
[Sender]   ACK 2 received
[Receiver] Frame 2 accepted, sends ACK 3
[Channel]  ACK 3 LOST in transit
[Receiver] Frame 3 accepted, sends ACK 4
[Sender]   ACK 4 received (cumulative) -> frames up to 3 confirmed,
           so the lost ACK does not matter
[Receiver] Frame 4 accepted, sends ACK 5
[Sender]   ACK 5 received
[Sender]   Window slides to base 5
------------------------------------------------------------
[Sender]   Window [5..7] -> sending frames: 5 6 7
[Receiver] Frame 5 accepted, sends ACK 6
[Sender]   ACK 6 received
[Receiver] Frame 6 accepted, sends ACK 7
[Sender]   ACK 7 received
[Receiver] Frame 7 accepted, sends ACK 8
[Sender]   ACK 8 received
[Sender]   Window slides to base 8
------------------------------------------------------------

All 7 frames delivered. Total transmissions = 7 (retransmissions = 0)
```

## Case 3b – Lost ACK for the last frame of the window

```
$ ./go_back_n
===== GO-BACK-N ARQ SIMULATION =====
1. Corrupted frame
2. Lost frame
3. Lost ACK
Enter your choice            : 3
Enter total number of frames : 6
Enter window size (N)        : 3
Enter the frame number for the error : 3

--- Case: LOST ACK at frame 3, window = 3 ---

[Sender]   Window [1..3] -> sending frames: 1 2 3
[Receiver] Frame 1 accepted, sends ACK 2
[Sender]   ACK 2 received
[Receiver] Frame 2 accepted, sends ACK 3
[Sender]   ACK 3 received
[Receiver] Frame 3 accepted, sends ACK 4
[Channel]  ACK 4 LOST in transit
[Sender]   TIMEOUT for frame 3 -> Go back to frame 3 and resend window
------------------------------------------------------------
[Sender]   Window [3..5] -> sending frames: 3 4 5
[Receiver] Frame 3 is a DUPLICATE -> discarded, re-sends ACK 4
[Receiver] Frame 4 accepted, sends ACK 5
[Sender]   ACK 5 received
[Receiver] Frame 5 accepted, sends ACK 6
[Sender]   ACK 6 received
[Sender]   Window slides to base 6
------------------------------------------------------------
[Sender]   Window [6..6] -> sending frames: 6
[Receiver] Frame 6 accepted, sends ACK 7
[Sender]   ACK 7 received
[Sender]   Window slides to base 7
------------------------------------------------------------

All 6 frames delivered. Total transmissions = 7 (retransmissions = 1)
```

*Output screenshot – Experiment 6*

(paste your own screenshot here, with your ID card visible)

| Case | What the receiver does | Sender recovery | Frames resent (run above) |
|---|---|---|---|
| Corrupted frame 3 | Discards F3 (bad check) and F4 (out of order) | Timeout → go back to F3 | F3, F4 (2) |
| Lost frame 3 | Never gets F3, discards F4 | Timeout → go back to F3 | F3, F4 (2) |
| Lost ACK of F2 | Accepted F2; later ACK 4 is cumulative | Nothing to do | none (0) |
| Lost ACK of last frame | Discards duplicate F3, re-ACKs | Timeout → resend from F3 | F3 (1) |

## Result

The Go-Back-N simulation delivered all frames in order in every case. A corrupted or lost frame 3 with window 4 caused frames 3 and 4 to be retransmitted (9 transmissions for 7 frames). A lost ACK in mid-window needed no retransmission because the next cumulative ACK confirmed the frame, while a lost ACK at the end of a window caused one duplicate retransmission.

## Conclusion

Go-Back-N keeps the link busy by pipelining up to N frames and needs only a one-frame receiver buffer, but one error forces retransmission of every frame sent after it, which wastes bandwidth on noisy links. Cumulative ACKs make it tolerant of lost ACKs.

EXPERIMENT
07

# Flow Control using Selective Repeat ARQ

*Language: C*

Date: _____ Name: _____ Reg. No: _____

## Aim

To simulate the Selective Repeat ARQ protocol in C, in which the receiver buffers out-of-order frames and the sender retransmits only the frame that was lost or corrupted.

## Theory

Selective Repeat uses a sending window and a **receiving window of the same size N**. With m-bit sequence numbers, $N \le 2^{m-1}$ so that new and old frames can never be confused. Each frame is acknowledged individually and has its own timer.

- Frames that arrive out of order but inside the receive window are **buffered**, not discarded.
- When the receiver notices a gap, it sends a **NAK** (negative acknowledgement) for the missing frame; if no later frame arrives, the sender's timer for that frame expires instead.
- The sender retransmits **only** the missing frame. The receiver then delivers it together with the buffered frames, in order, to the network layer.
- It uses bandwidth better than Go-Back-N at the cost of receiver buffers and more complex logic.

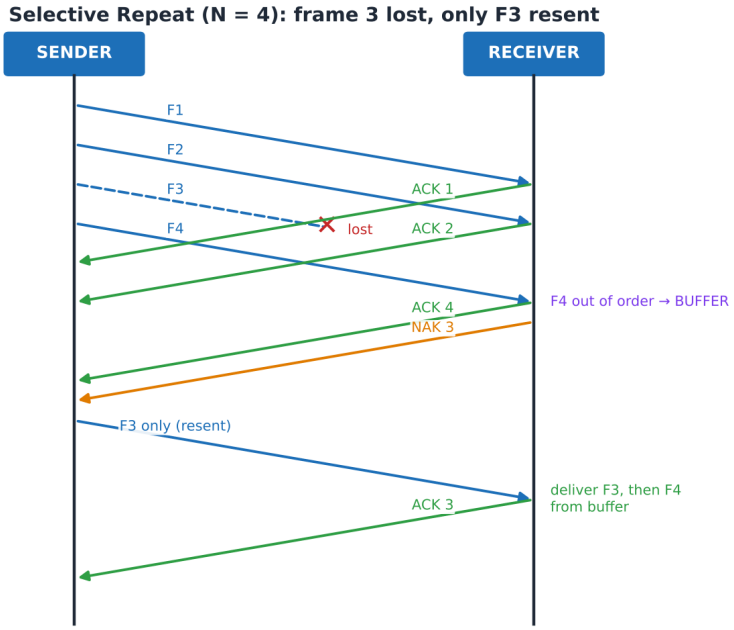



*Figure 7.1 – Selective Repeat: F4 is buffered, NAK 3 triggers retransmission of F3 only*

| | Stop-and-Wait | Go-Back-N | Selective Repeat |
|---|---|---|---|
| Sender window | 1 | $N \le 2^m - 1$ | $N \le 2^{m-1}$ |
| Receiver window | 1 | 1 | N |
| Out-of-order frames | – | Discarded | Buffered |
| On error resend | That frame | That frame and all after it | Only that frame |
| ACK type | Individual | Cumulative | Individual + NAK |
| Complexity | Lowest | Medium | Highest |

## Algorithm

1. Start.
2. Read total frames, window size N, error type (lost / corrupted) and the error frame.
3. base = 1 (sender), rbase = 1 (next frame the receiver must deliver); all acked[] and buffered[] = 0.
4. Send every frame in base … base + N − 1 that is not yet acknowledged.
5. For each frame: if it is the lost/corrupted one, drop it. Otherwise, if frame = rbase deliver it, increment rbase and deliver any consecutive buffered frames; else store it in the buffer. Send an individual ACK and mark it acknowledged.
6. If a frame was dropped: send NAK (if a later frame arrived) or let its timer expire, then retransmit only that frame; the receiver delivers it and the buffered frames in order.
7. Slide base past all consecutively acknowledged frames.
8. Repeat while base ≤ total; display total transmissions.
9. Stop.

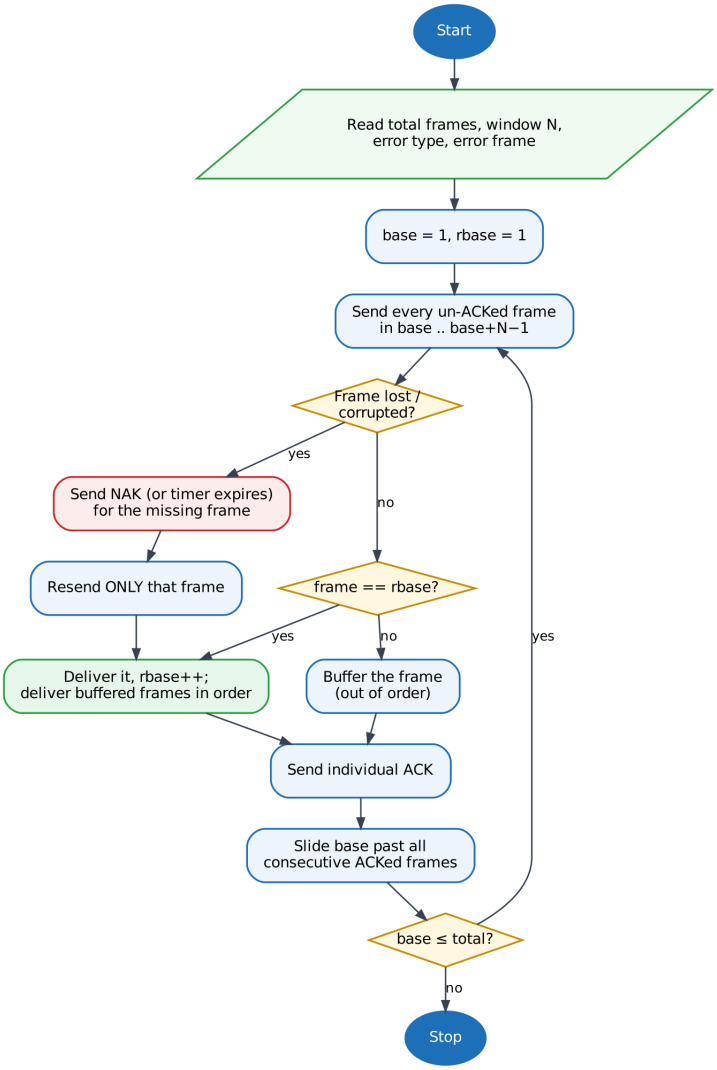



*Figure 7.2 – Flowchart of the Selective Repeat program*

## Program

```
/* Experiment 7: Flow control using Selective Repeat ARQ
   Receiver buffers out-of-order frames; sender retransmits ONLY the
   frame that was lost/corrupted (triggered by NAK or by its own timer). */
#include <stdio.h>

int main(void)
{
    int total, N, err, choice;
    int acked[100] = {0}, buffered[100] = {0};
    int base = 1, rbase = 1, transmissions = 0, done = 0;
```

```
    printf("===== SELECTIVE REPEAT ARQ SIMULATION =====\n");
    printf("Enter total number of frames : ");
    scanf("%d", &total);
    printf("Enter window size (N)        : ");
    scanf("%d", &N);
    printf("Error type (1 = lost, 2 = corrupted) : ");
    scanf("%d", &choice);
    printf("Enter the frame number for the error : ");
    scanf("%d", &err);

    while (base <= total) {
        int end = base + N - 1, bad = 0, later_arrived = 0;
        if (end > total) end = total;

        printf("\n[Sender]   Window [%d..%d] -> sending un-ACKed frames:", base, end);
        for (int f = base; f <= end; f++)
            if (!acked[f]) printf(" %d", f);
        printf("\n");

        for (int f = base; f <= end; f++) {
            if (acked[f]) continue;
            transmissions++;
            if (!done && f == err) {
                if (choice == 1) printf("[Channel]  Frame %d LOST\n", f);
                else             printf("[Receiver] Frame %d CORRUPTED -> discarded\n", f);
                done = 1;
                bad = f;
                continue;
            }
            if (bad) later_arrived = 1;
            if (f == rbase) {
                printf("[Receiver] Frame %d received in order -> delivered, sends ACK %d\n", f, f);
                rbase++;
                while (buffered[rbase]) {                /* release buffered frames */
                    printf("[Receiver] Buffered frame %d delivered in order\n", rbase);
                    buffered[rbase] = 0;
                    rbase++;
                }
            } else {
                buffered[f] = 1;
                printf("[Receiver] Frame %d out of order -> BUFFERED, sends ACK %d\n", f, f);
            }
            acked[f] = 1;
        }

        if (bad) {
            if (later_arrived)
                printf("[Receiver] Gap detected -> sends NAK %d\n"
                       "[Sender]   NAK %d received -> retransmitting ONLY frame %d\n", bad, bad, bad);
            else
                printf("[Sender]   Timer for frame %d expired -> retransmitting ONLY frame %d\n",
                       bad, bad);
            transmissions++;
            printf("[Receiver] Frame %d received -> delivered, sends ACK %d\n", bad, bad);
            acked[bad] = 1;
            rbase++;
            while (buffered[rbase]) {
                printf("[Receiver] Buffered frame %d delivered in order\n", rbase);
                buffered[rbase] = 0;
                rbase++;
            }
        }

        while (base <= total && acked[base]) base++;
        printf("[Sender]   Window slides, new base = %d\n", base);
        printf("------------------------------------------------------------");
    }
    printf("\n\nAll %d frames delivered in order. Total transmissions = %d (retransmissions = %d)\n",
           total, transmissions, transmissions - total);
    return 0;
}
```

## How to run

- Code::Blocks: *File* → *New* → *Empty File*, paste the program, save it as **selective_repeat.c**, then press **F9** (Build and Run).
- Terminal: `gcc selective_repeat.c -o selective_repeat` then `./selective_repeat` (on Windows: `selective_repeat.exe`).

- Inputs used below: 8 frames, window 4, frame 3 lost; and 6 frames, window 3, frame 2 corrupted.

## Output

### Case 1 – Frame 3 lost

```
$ ./selective_repeat
===== SELECTIVE REPEAT ARQ SIMULATION =====
Enter total number of frames : 8
Enter window size (N)        : 4
Error type (1 = lost, 2 = corrupted) : 1
Enter the frame number for the error : 3

[Sender]   Window [1..4] -> sending un-ACKed frames: 1 2 3 4
[Receiver] Frame 1 received in order -> delivered, sends ACK 1
[Receiver] Frame 2 received in order -> delivered, sends ACK 2
[Channel]  Frame 3 LOST
[Receiver] Frame 4 out of order -> BUFFERED, sends ACK 4
[Receiver] Gap detected -> sends NAK 3
[Sender]   NAK 3 received -> retransmitting ONLY frame 3
[Receiver] Frame 3 received -> delivered, sends ACK 3
[Receiver] Buffered frame 4 delivered in order
[Sender]   Window slides, new base = 5
------------------------------------------------------------
[Sender]   Window [5..8] -> sending un-ACKed frames: 5 6 7 8
[Receiver] Frame 5 received in order -> delivered, sends ACK 5
[Receiver] Frame 6 received in order -> delivered, sends ACK 6
[Receiver] Frame 7 received in order -> delivered, sends ACK 7
[Receiver] Frame 8 received in order -> delivered, sends ACK 8
[Sender]   Window slides, new base = 9
------------------------------------------------------------

All 8 frames delivered in order. Total transmissions = 9 (retransmissions = 1)
```

### Case 2 – Frame 2 corrupted

```
$ ./selective_repeat
===== SELECTIVE REPEAT ARQ SIMULATION =====
Enter total number of frames : 6
Enter window size (N)        : 3
Error type (1 = lost, 2 = corrupted) : 2
Enter the frame number for the error : 2

[Sender]   Window [1..3] -> sending un-ACKed frames: 1 2 3
[Receiver] Frame 1 received in order -> delivered, sends ACK 1
[Receiver] Frame 2 CORRUPTED -> discarded
[Receiver] Frame 3 out of order -> BUFFERED, sends ACK 3
[Receiver] Gap detected -> sends NAK 2
[Sender]   NAK 2 received -> retransmitting ONLY frame 2
[Receiver] Frame 2 received -> delivered, sends ACK 2
[Receiver] Buffered frame 3 delivered in order
[Sender]   Window slides, new base = 4
------------------------------------------------------------
[Sender]   Window [4..6] -> sending un-ACKed frames: 4 5 6
[Receiver] Frame 4 received in order -> delivered, sends ACK 4
[Receiver] Frame 5 received in order -> delivered, sends ACK 5
[Receiver] Frame 6 received in order -> delivered, sends ACK 6
[Sender]   Window slides, new base = 7
------------------------------------------------------------

All 6 frames delivered in order. Total transmissions = 7 (retransmissions = 1)
```

*Output screenshot – Experiment 7*

(paste your own screenshot here, with your ID card visible)

## Result

Selective Repeat delivered all frames in order. When frame 3 was lost, frame 4 was buffered, NAK 3 was sent and only frame 3 was retransmitted (9 transmissions for 8 frames, compared with extra resends of later frames in Go-Back-N).

## Conclusion

Selective Repeat ARQ gives the best link utilisation of the three ARQ schemes because it retransmits only the erroneous frames, at the cost of buffering at the receiver and a smaller maximum window (half the sequence space).

EXPERIMENT
**08**

# Implement IP Addressing and Default IP Gateway using CPT

*Tool: Cisco Packet Tracer*

Date: _____ Name: _____ Reg. No: _____

## Aim

To design a network of two LANs connected by a router in Cisco Packet Tracer, assign IPv4 addresses, subnet masks and default gateways to all devices, and verify that hosts in different subnets can communicate only when the default gateway is configured.

## Theory

An **IPv4 address** (32 bits, dotted decimal) has a network part and a host part, separated by the **subnet mask**. 192.168.10.10/24 means the first 24 bits (192.168.10) identify the network and the last 8 bits (.10) identify the host; the network address is 192.168.10.0 and the broadcast address is 192.168.10.255, leaving 254 usable host addresses.

A host compares the destination address with its own network. If the destination is on the same network, it sends the frame directly (after ARP). If not, it sends the packet to its **default gateway** – the IP address of the router interface on its own LAN – and the router forwards it. Without a gateway a host can reach only its own subnet.

Private address ranges (RFC 1918): 10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16.

## Requirements

- Cisco Packet Tracer 8.x
- 1 × Cisco 2911 router, 2 × 2960-24TT switches, 4 × PC-PT
- Copper straight-through cables

## Network topology

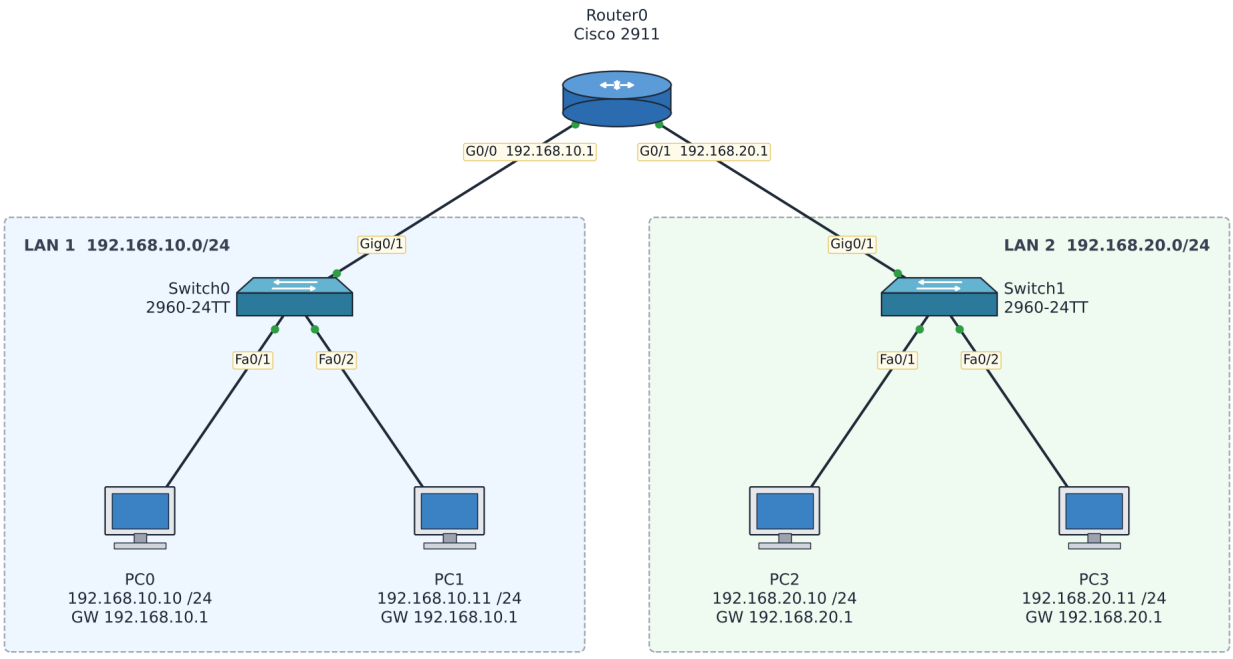



*Figure 8.1 – Two LANs joined by Router0*

| Device | Interface | IP address | Subnet mask | Default gateway |
|---|---|---|---|---|
| Router0 | G0/0 | 192.168.10.1 | 255.255.255.0 | – |
| Router0 | G0/1 | 192.168.20.1 | 255.255.255.0 | – |
| PC0 | Fa0 | 192.168.10.10 | 255.255.255.0 | 192.168.10.1 |
| PC1 | Fa0 | 192.168.10.11 | 255.255.255.0 | 192.168.10.1 |
| PC2 | Fa0 | 192.168.20.10 | 255.255.255.0 | 192.168.20.1 |
| PC3 | Fa0 | 192.168.20.11 | 255.255.255.0 | 192.168.20.1 |

## Procedure

### Step 1. Place the devices

Open Cisco Packet Tracer → *File* → *New*. From the device-type selection box at the bottom-left drag the following onto the Logical workspace (see the “Packet Tracer basics” section, Figure B.1):

- Network Devices → Routers → **2911** (Router0)
- Network Devices → Switches → **2960** × 2 (Switch0, Switch1)
- End Devices → **PC** × 4 (PC0 – PC3)

Click each device and change its **Display Name** (Config tab → Global → Settings) to match the diagram, so the labels in your screenshots match the procedure.

### Step 2. Connect the cables

Click the **Connections** (lightning-bolt) icon. Choose the cable type, click the first device, pick the port from the pop-up list, then click the second device and pick its port:

- Copper Straight-Through: Router0 **GigabitEthernet0/0** → Switch0 GigabitEthernet0/1
- Copper Straight-Through: Router0 **GigabitEthernet0/1** → Switch1 GigabitEthernet0/1
- Copper Straight-Through: PC0, PC1 FastEthernet0 → Switch0 Fa0/1, Fa0/2
- Copper Straight-Through: PC2, PC3 FastEthernet0 → Switch1 Fa0/1, Fa0/2

Wait until the link lights turn **green** (switch ports take about 30 s to move from amber to green). Router ports stay **red** until you enter `no shutdown`.

### Step 3. Configure the router interfaces

Click Router0 → **CLI** tab (Figure B.3). Answer **no** to the initial configuration dialog and press Enter, then type:

```
Router> enable
Router# configure terminal
Router(config)# hostname R0
R0(config)# interface GigabitEthernet0/0
R0(config-if)# ip address 192.168.10.1 255.255.255.0
R0(config-if)# no shutdown
R0(config-if)# exit
R0(config)# interface GigabitEthernet0/1
R0(config-if)# ip address 192.168.20.1 255.255.255.0
R0(config-if)# no shutdown
R0(config-if)# end
R0# copy running-config startup-config
```

(Alternative without CLI: Config tab → INTERFACE → GigabitEthernet0/0 → tick **On** and type the address and mask.) The link lights on the router side turn green.

### Step 4. Configure the end devices (static IP)

Click a PC → **Desktop** tab → **IP Configuration** → select **Static** and type the values from the table (Figure B.2). For a server use *Config* → *FastEthernet0* for the address and *Config* → *Global* → *Settings* for the gateway and DNS.

| Device | IPv4 Address | Subnet Mask | Default Gateway | DNS Server |
|---|---|---|---|---|
| PC0 | 192.168.10.10 | 255.255.255.0 | 192.168.10.1 | – |
| PC1 | 192.168.10.11 | 255.255.255.0 | 192.168.10.1 | – |

| Device | IPv4 Address | Subnet Mask | Default Gateway | DNS Server |
|---|---|---|---|---|
| PC2 | 192.168.20.10 | 255.255.255.0 | 192.168.20.1 | – |
| PC3 | 192.168.20.11 | 255.255.255.0 | 192.168.20.1 | – |

### Step 5. Verify the router

In the router CLI:

```
R0# show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.10.1    YES manual up                    up
GigabitEthernet0/1     192.168.20.1    YES manual up                    up
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
```

Both interfaces must show **up / up**. `show ip route` shows two C (connected) and two L (local) routes.

### Step 6. Test connectivity

PC0 → Desktop → **Command Prompt**. Run `ipconfig`, ping the gateway, a PC on the same LAN and a PC on the other LAN, and trace the path:

```
C:\> ping 192.168.10.1        (own gateway)
C:\> ping 192.168.10.11       (same subnet)
C:\> ping 192.168.20.10       (other subnet, through the router)
C:\> tracert 192.168.20.10
```

**Expected output** (the first reply may time out while ARP resolves the MAC address):

```
Pinging 192.168.20.10 with 32 bytes of data:

Request timed out.
Reply from 192.168.20.10: bytes=32 time<1ms TTL=127
Reply from 192.168.20.10: bytes=32 time<1ms TTL=127
Reply from 192.168.20.10: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.20.10:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),

C:\> tracert 192.168.20.10
  1   0 ms   0 ms   0 ms   192.168.10.1
  2   0 ms   0 ms   0 ms   192.168.20.10
Trace complete.
```

TTL = 127 (one less than 128) shows the packet passed through one router.

### Step 7. Demonstrate the role of the default gateway

On PC0 clear the Default Gateway field (set it to 0.0.0.0). Ping 192.168.10.11 again – it still works (same subnet). Ping 192.168.20.10 – it now fails with *Request timed out*, because PC0 has no router to send off-subnet traffic to. Restore 192.168.10.1 and the ping works again.

### Step 8. Optional – watch the packet

Switch to **Simulation** mode, add a simple PDU (closed-envelope icon) from PC0 to PC2 and press **Capture/Forward**: the ICMP packet goes PC0 → Switch0 → Router0 → Switch1 → PC2. Click the envelope at Router0 to see that the source/destination IPs stay the same while the MAC addresses change at each hop.

*Screenshot – topology with all links green*

(paste your own screenshot here, with your ID card visible)

*Screenshot – successful ping / tracert from PC0 to PC2*

(paste your own screenshot here, with your ID card visible)

## Result

Two subnets (192.168.10.0/24 and 192.168.20.0/24) were configured with static IP addresses and default gateways. Hosts on the same subnet communicated directly, and hosts on different subnets communicated through Router0; removing the default gateway on PC0 broke only inter-subnet communication.

## Conclusion

Correct IP addressing places each host in the right network, and the default gateway is the host’s exit point to every other network; the router connects the subnets by forwarding packets between its directly connected interfaces.

EXPERIMENT
**09**

# Implement RIP and OSPF using CPT

*Tool: Cisco Packet Tracer*

Date: _____ Name: _____ Reg. No: _____

## Aim

To configure the RIP (version 2) and OSPF dynamic routing protocols on two routers in Cisco Packet Tracer, verify the learned routes in the routing tables, and compare the two protocols.

## Theory

**RIP (Routing Information Protocol)** is a *distance-vector* protocol. The metric is the **hop count** (maximum 15; 16 = unreachable). Every 30 s each router sends its whole routing table to its neighbours (RIPv2 uses multicast 224.0.0.9 and carries subnet masks). Administrative distance = 120.

**OSPF (Open Shortest Path First)** is a *link-state* protocol. Routers form neighbour adjacencies using Hello packets, flood Link-State Advertisements (LSAs) so that every router in an area has the same map of the network, and each runs **Dijkstra's SPF algorithm** to build its routing table. The metric is **cost** = 100 Mbps / interface bandwidth (1 for FastEthernet and GigabitEthernet by default). Administrative distance = 110. OSPF uses wildcard masks in its network statements and areas for hierarchy.

## Requirements

- 2 × Cisco 2911 routers, 2 × 2960 switches, 4 × PC-PT
- Copper straight-through cables (copper cross-over for router–router is also accepted; Gigabit ports auto-detect)

## Network topology

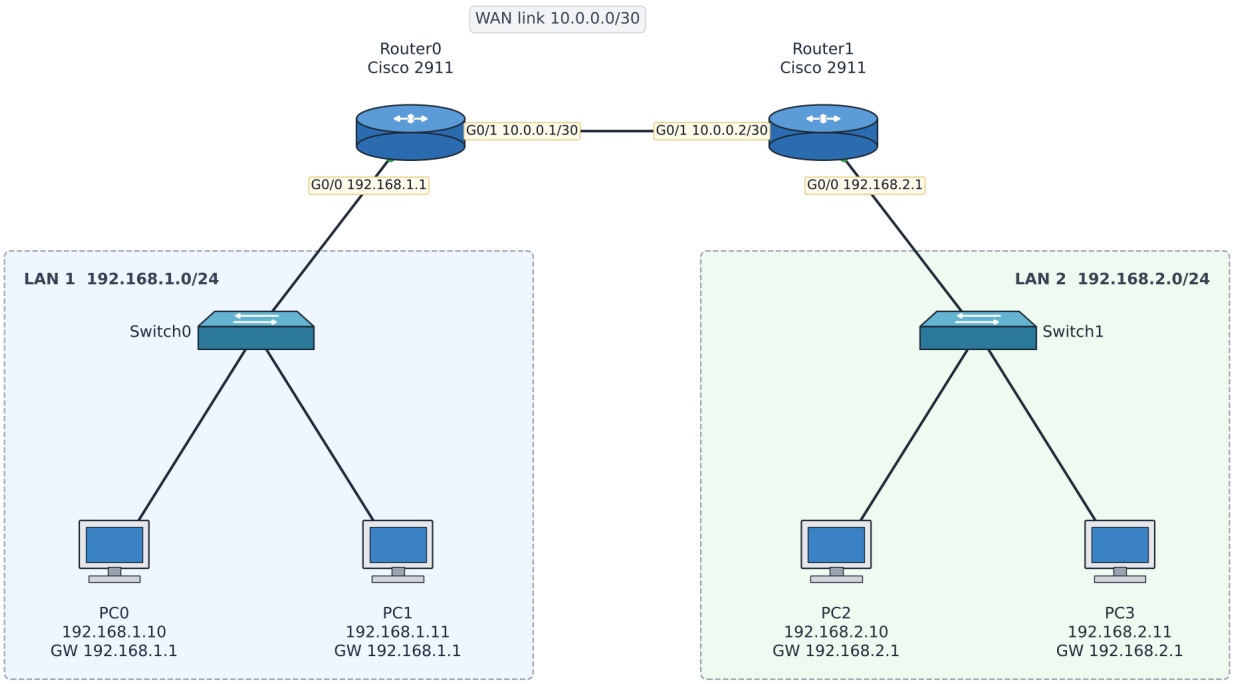



*Figure 9.1 – Two LANs connected through a /30 WAN link*

| Device | Interface | IP address / mask | Gateway |
|---|---|---|---|
| Router0 | G0/0 | `192.168.1.1 /24` | – |

| Device | Interface | IP address / mask | Gateway |
|---|---|---|---|
| Router0 | G0/1 | `10.0.0.1 /30` | – |
| Router1 | G0/0 | `192.168.2.1 /24` | – |
| Router1 | G0/1 | `10.0.0.2 /30` | – |
| PC0 / PC1 | Fa0 | `192.168.1.10 / .11 /24` | `192.168.1.1` |
| PC2 / PC3 | Fa0 | `192.168.2.10 / .11 /24` | `192.168.2.1` |

## Procedure

### Part A – Build the network

#### Step 1. Place the devices

Open Cisco Packet Tracer → *File* → *New*. From the device-type selection box at the bottom-left drag the following onto the Logical workspace (see the “Packet Tracer basics” section, Figure B.1):

- 2 × 2911 routers (Router0, Router1)
- 2 × 2960 switches
- 4 × PCs

Click each device and change its **Display Name** (Config tab → Global → Settings) to match the diagram, so the labels in your screenshots match the procedure.

#### Step 2. Connect the cables

Click the **Connections** (lightning-bolt) icon. Choose the cable type, click the first device, pick the port from the pop-up list, then click the second device and pick its port:

- Router0 G0/1 ↔ Router1 G0/1 (copper straight-through or cross-over)
- Router0 G0/0 → Switch0 G0/1; Router1 G0/0 → Switch1 G0/1
- PC0, PC1 → Switch0; PC2, PC3 → Switch1

Wait until the link lights turn **green** (switch ports take about 30 s to move from amber to green). Router ports stay **red** until you enter `no shutdown`.

#### Step 3. Configure the router interfaces

Router0 CLI:

```
Router> enable
Router# configure terminal
Router(config)# hostname R0
R0(config)# interface g0/0
R0(config-if)# ip address 192.168.1.1 255.255.255.0
R0(config-if)# no shutdown
R0(config-if)# interface g0/1
R0(config-if)# ip address 10.0.0.1 255.255.255.252
R0(config-if)# no shutdown
R0(config-if)# end
```

Router1 CLI:

```
Router(config)# hostname R1
R1(config)# interface g0/0
R1(config-if)# ip address 192.168.2.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# interface g0/1
R1(config-if)# ip address 10.0.0.2 255.255.255.252
R1(config-if)# no shutdown
R1(config-if)# end
```

#### Step 4. Configure the end devices (static IP)

Click a PC → **Desktop** tab → **IP Configuration** → select **Static** and type the values from the table (Figure B.2). For a server use *Config* → *FastEthernet0* for the address and *Config* → *Global* → *Settings* for the gateway and DNS.

| Device | IPv4 Address | Subnet Mask | Default Gateway | DNS Server |
|---|---|---|---|---|
| PC0 | 192.168.1.10 | 255.255.255.0 | 192.168.1.1 | – |
| PC1 | 192.168.1.11 | 255.255.255.0 | 192.168.1.1 | – |
| PC2 | 192.168.2.10 | 255.255.255.0 | 192.168.2.1 | – |
| PC3 | 192.168.2.11 | 255.255.255.0 | 192.168.2.1 | – |

### Step 5. Check that routing is needed

From PC0 ping 192.168.2.10 – it **fails**. `show ip route` on R0 lists only 192.168.1.0 and 10.0.0.0 (connected): R0 does not know where 192.168.2.0 is yet.

## Part B – Configure RIP version 2

### Step 6. Enable RIP on both routers

Advertise every directly connected network (RIP network statements are classful):

```
R0(config)# router rip
R0(config-router)# version 2
R0(config-router)# no auto-summary
R0(config-router)# network 192.168.1.0
R0(config-router)# network 10.0.0.0
R0(config-router)# end

R1(config)# router rip
R1(config-router)# version 2
R1(config-router)# no auto-summary
R1(config-router)# network 192.168.2.0
R1(config-router)# network 10.0.0.0
R1(config-router)# end
```

(GUI alternative: Config tab → ROUTING → RIP → type the network → Add. This configures RIP v1, so the CLI is preferred.)

### Step 7. Verify RIP

Wait about 30 s, then on R0:

```
R0# show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area ...

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.0.0.0/30 is directly connected, GigabitEthernet0/1
L       10.0.0.1/32 is directly connected, GigabitEthernet0/1
     192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.1.0/24 is directly connected, GigabitEthernet0/0
L       192.168.1.1/32 is directly connected, GigabitEthernet0/0
R    192.168.2.0/24 [120/1] via 10.0.0.2, 00:00:12, GigabitEthernet0/1
```

The **R** route [120/1] means administrative distance 120 and metric 1 hop. Also try `show ip protocols` (update timer 30 s, invalid 180 s, flush 240 s) and `debug ip rip` (then `undebug all`). Ping PC0 → PC2: replies with TTL = 126 (two routers).

## Part C – Replace RIP with OSPF

### Step 8. Remove RIP and configure OSPF

```
R0(config)# no router rip
R0(config)# router ospf 1
R0(config-router)# router-id 1.1.1.1
R0(config-router)# network 192.168.1.0 0.0.0.255 area 0
R0(config-router)# network 10.0.0.0 0.0.0.3 area 0
R0(config-router)# end

R1(config)# no router rip
R1(config)# router ospf 1
R1(config-router)# router-id 2.2.2.2
R1(config-router)# network 192.168.2.0 0.0.0.255 area 0
R1(config-router)# network 10.0.0.0 0.0.0.3 area 0
R1(config-router)# end
```

The second number in each network line is the **wildcard mask** (inverse of the subnet mask: 255.255.255.252 → 0.0.0.3). When the adjacency forms you will see `%OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/1 from LOADING to FULL, Loading Done`.

### Step 9. Verify OSPF

```
R0# show ip ospf neighbor
Neighbor ID     Pri   State           Dead Time   Address         Interface
2.2.2.2           1   FULL/DR         00:00:35    10.0.0.2        GigabitEthernet0/1

R0# show ip route ospf
O    192.168.2.0/24 [110/2] via 10.0.0.2, 00:01:05, GigabitEthernet0/1
```

The route is now marked **O** with [110/2]: distance 110, cost 2 (WAN link cost 1 + LAN cost 1). Ping PC0 → PC2 and PC1 → PC3 again – all succeed. Save with `write memory` on both routers.

### Step 10. Trace the packet in Simulation mode

Simulation mode → Edit Filters → keep only ICMP (and RIP/OSPF to see routing updates). Send a PDU from PC0 to PC2 and open it at Router0: the OSI model tab shows the decision “the routing table has a matching entry”. With the OSPF filter on you can also see periodic OSPF Hello packets every 10 s.

*Screenshot – show ip route with R routes (RIP) and ping*

(paste your own screenshot here, with your ID card visible)

*Screenshot – show ip route / show ip ospf neighbor with O routes (OSPF)*

(paste your own screenshot here, with your ID card visible)

### Comparison observed

| Feature | RIP v2 | OSPF |
|---|---|---|
| Type | Distance vector (Bellman-Ford) | Link state (Dijkstra SPF) |
| Metric | Hop count (max 15) | Cost = $10^8$ / bandwidth |
| Updates | Whole table every 30 s | Hello every 10 s; LSAs only on change |
| Admin distance | 120 | 110 |
| Route code | R | O |
| Convergence / scale | Slow, small networks | Fast, large hierarchical networks (areas) |

## Result

RIP v2 and then OSPF were configured on Router0 and Router1. With RIP, each router learned the remote LAN as an R route with metric 1 hop; after replacing RIP with OSPF the neighbours reached the FULL state and the same LAN appeared as an O route with cost 2. End-to-end pings between the two LANs succeeded under both protocols.

## Conclusion

Both protocols automatically build routing tables. RIP is very easy to configure but uses only hop count and converges slowly; OSPF uses bandwidth-based cost, sends updates only on change and converges faster, which makes it the better choice for larger networks.

EXPERIMENT 10

# Implement Communication using SMTP and DNS using CPT

*Tool: Cisco Packet Tracer*

Date: _____ Name: _____ Reg. No: _____

## Aim

To build a LAN in Cisco Packet Tracer with a server providing DNS, SMTP and POP3 services, resolve domain names with DNS, and send and receive e-mail between client PCs.

## Theory

- **DNS (Domain Name System)** translates host names such as mail.cnlab.com into IP addresses. Clients send queries to the DNS server on **UDP port 53**; an **A record** maps a name to an IPv4 address.
- **SMTP (Simple Mail Transfer Protocol)**, **TCP port 25**, is the *push* protocol used by a mail client to send mail to the mail server (and between servers).
- **POP3 (Post Office Protocol v3)**, **TCP port 110**, is the *pull* protocol the recipient's client uses to download mail from its mailbox on the server.
- Flow: PC0 resolves mail.cnlab.com via DNS → sends the mail with SMTP → server stores it in user2's mailbox → PC1 resolves the name → retrieves the mail with POP3.

## Requirements

- 1 × Server-PT, 1 × 2960 switch, 3 × PC-PT, copper straight-through cables

## Network topology

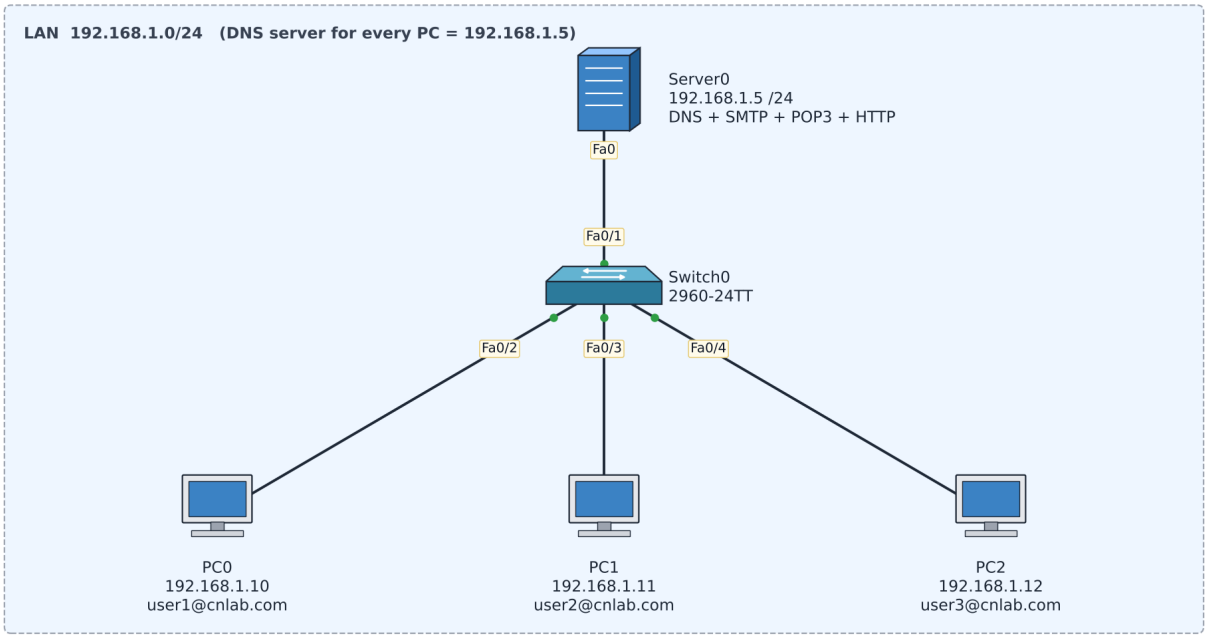



*Figure 10.1 – Mail and DNS server on a single LAN*

## Procedure

### Step 1. Place the devices

Open Cisco Packet Tracer → *File* → *New*. From the device-type selection box at the bottom-left drag the following onto the Logical workspace (see the "Packet Tracer basics" section, Figure B.1):

- End Devices → **Server** (Server0)

- Network Devices → Switches → 2960 (Switch0)
- End Devices → PC × 3 (PC0, PC1, PC2)

  Click each device and change its **Display Name** (Config tab → Global → Settings) to match the diagram, so the labels in your screenshots match the procedure.

### Step 2. Connect the cables

Click the **Connections** (lightning-bolt) icon. Choose the cable type, click the first device, pick the port from the pop-up list, then click the second device and pick its port:

- Server0 FastEthernet0 → Switch0 Fa0/1
- PC0, PC1, PC2 FastEthernet0 → Switch0 Fa0/2 – Fa0/4

  Wait until the link lights turn **green** (switch ports take about 30 s to move from amber to green). Router ports stay **red** until you enter `no shutdown`.

### Step 3. Configure the server's IP address

Server0 → **Config** → **FastEthernet0**: IP 192.168.1.5, mask 255.255.255.0. Config → **Global** → **Settings**: Gateway 192.168.1.1 (not used on one LAN, but harmless), DNS Server 192.168.1.5.

### Step 4. Configure the end devices (static IP)

Click a PC → **Desktop** tab → **IP Configuration** → select **Static** and type the values from the table (Figure B.2). For a server use *Config* → *FastEthernet0* for the address and *Config* → *Global* → *Settings* for the gateway and DNS. The **DNS Server** field must point to 192.168.1.5 on every PC, otherwise names cannot be resolved.

| Device | IPv4 Address | Subnet Mask | Default Gateway | DNS Server |
|---|---|---|---|---|
| PC0 | 192.168.1.10 | 255.255.255.0 | 192.168.1.1 | 192.168.1.5 |
| PC1 | 192.168.1.11 | 255.255.255.0 | 192.168.1.1 | 192.168.1.5 |
| PC2 | 192.168.1.12 | 255.255.255.0 | 192.168.1.1 | 192.168.1.5 |

### Step 5. Turn on DNS and add records

Server0 → **Services** → **DNS** (Figure 10.2). Set DNS Service to **On**, then add these A records one at a time (type Name, choose A Record, type Address, click **Add**):

| Name | Type | Address |
|---|---|---|
| mail.cnlab.com | A Record | 192.168.1.5 |
| www.cnlab.com | A Record | 192.168.1.5 |

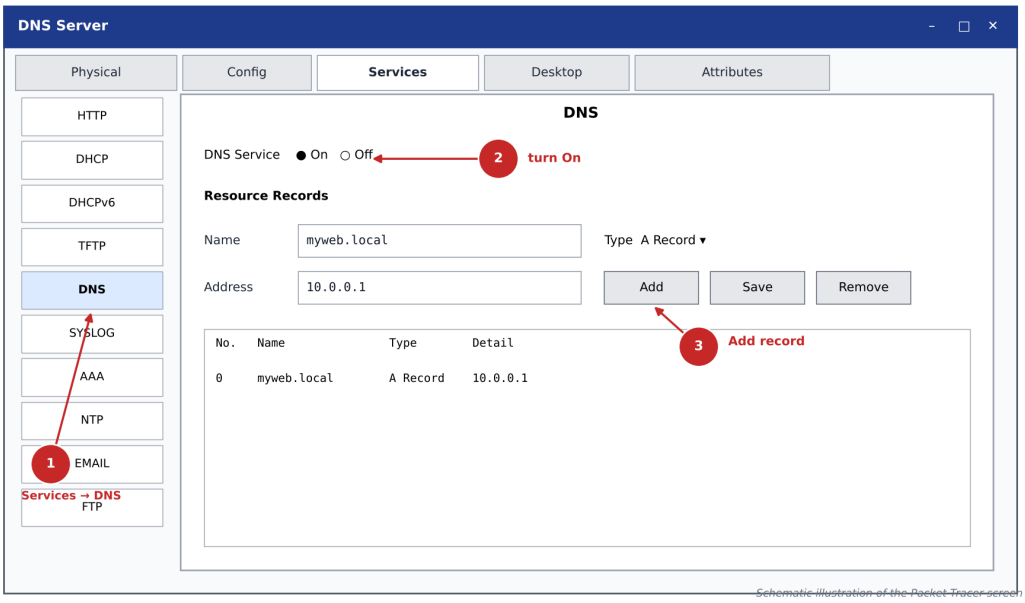



*Figure 10.2 – Services → DNS: turning the service on and adding an A record*

### Step 6. Turn on SMTP and POP3 and create mailboxes

Server0 → Services → **EMAIL**. Set **SMTP Service: On** and **POP3 Service: On**. In **Domain Name** type `cnlab.com` and click **Set**. Under User Setup add the accounts (User + Password → **+**):

| User | Password | E-mail address |
|---|---|---|
| user1 | 1234 | user1@cnlab.com |
| user2 | 1234 | user2@cnlab.com |
| user3 | 1234 | user3@cnlab.com |

The domain after @ in each address must match the domain name set on the server.

### Step 7. Configure the mail client on each PC

PC0 → Desktop → **Email**. Fill in the Configure Mail form (Figure 10.3) and click **Save**. Repeat on PC1 (user2) and PC2 (user3).

| Field | PC0 | PC1 | PC2 |
|---|---|---|---|
| Your Name | user1 | user2 | user3 |
| Email Address | user1@cnlab.com | user2@cnlab.com | user3@cnlab.com |
| Incoming Mail Server | mail.cnlab.com | mail.cnlab.com | mail.cnlab.com |
| Outgoing Mail Server | mail.cnlab.com | mail.cnlab.com | mail.cnlab.com |
| User Name / Password | user1 / 1234 | user2 / 1234 | user3 / 1234 |

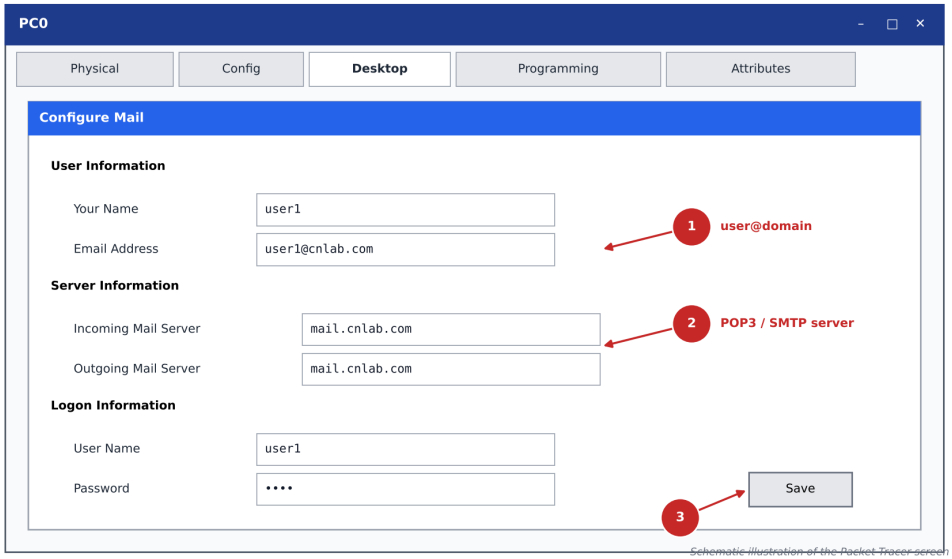



*Figure 10.3 – Desktop → Email → Configure Mail*

### Step 8. Verify DNS name resolution

PC0 → Command Prompt:

```
C:\> nslookup mail.cnlab.com

Server: [192.168.1.5]
Address:  192.168.1.5

Non-authoritative answer:
Name:   mail.cnlab.com
Address:  192.168.1.5

C:\> ping www.cnlab.com
Pinging 192.168.1.5 with 32 bytes of data:
Reply from 192.168.1.5: bytes=32 time<1ms TTL=128 ...
```

The ping by *name* works only because DNS translated it to 192.168.1.5. (Optional: Desktop → Web Browser → http://www.cnlab.com shows the server's default HTTP page.)

### Step 9. Send and receive e-mail

On PC0 → Email → **Compose**: To = user2@cnlab.com, Subject = Lab test, type a message and click **Send**. The status bar shows *Send Success*. On PC1 → Email → **Receive**: the message from user1@cnlab.com appears in the inbox. Reply from PC1 and receive it on PC0 to test both directions.

### Step 10. Watch the protocols in Simulation mode

Simulation → Edit Filters → tick only **DNS**, **SMTP** and **POP3**. Send another mail and press Capture/Forward. Observe: a DNS query/response (UDP 53) for mail.cnlab.com, then the SMTP exchange (TCP 25) from PC0 to the server; when PC1 clicks Receive, a DNS query and then POP3 (TCP 110). Click an envelope → Inbound PDU Details to see the port numbers.

*Screenshot – nslookup / ping by name*

(paste your own screenshot here, with your ID card visible)

*Screenshot – mail sent from PC0 and received on PC1*

(paste your own screenshot here, with your ID card visible)

## Result

The server was configured with DNS records for mail.cnlab.com and www.cnlab.com and with SMTP/POP3 mailboxes for three users. Clients resolved the names to 192.168.1.5, and e-mail sent from user1 on PC0 was delivered to and retrieved by user2 on PC1 (“Send Success” and the message visible in PC1’s inbox).

## Conclusion

The experiment showed how application-layer services cooperate: DNS (UDP 53) supplies the server address, SMTP (TCP 25) pushes mail to the server, and POP3 (TCP 110) lets the recipient pull it from the mailbox.

EXPERIMENT
**11**

# Implement Unicast Routing with the help of Link State Routing (LSR) using CPT

*Tool: Cisco Packet Tracer • Protocol: OSPF*

Date: _____ Name: _____ Reg. No: _____

## Aim

To implement link-state unicast routing using OSPF on a three-router network in Cisco Packet Tracer, examine the neighbour table, link-state database and shortest-path routes, and observe how the routers recompute paths when a link fails or its cost changes.

## Theory

In **link-state routing** every router learns the complete topology rather than only distances from neighbours. It works in four stages:

1. **Discover neighbours** – OSPF Hello packets (multicast 224.0.0.5, every 10 s; dead interval 40 s).
2. **Describe own links** – each router creates a Link-State Advertisement (LSA) listing its links and their costs.
3. **Flood** – LSAs are flooded to all routers in the area so every router holds an identical **Link-State Database (LSDB)**.
4. **Compute** – each router runs **Dijkstra's shortest path first (SPF)** algorithm with itself as root and installs the least-cost path to each network.

When a link changes, only a new LSA is flooded and SPF is re-run, so convergence is fast and there are no routing loops of the count-to-infinity kind seen with distance-vector protocols.

## Network topology

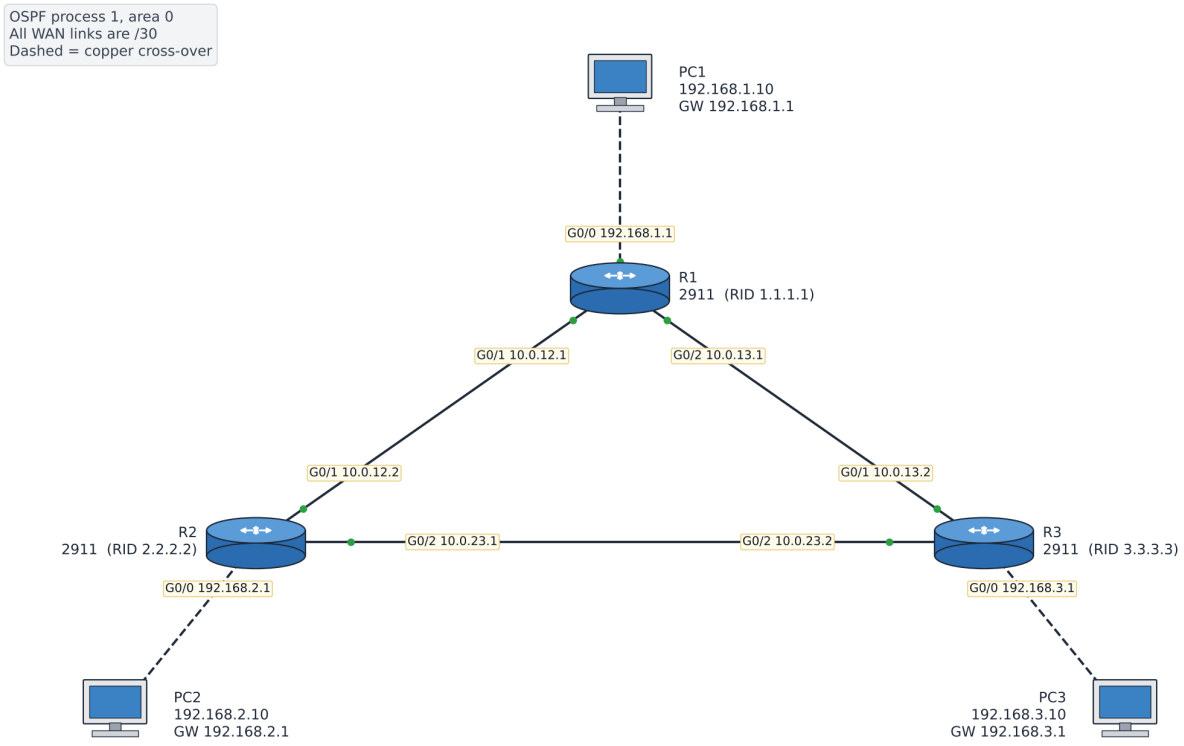



*Figure 11.1 – Triangle of three routers, one LAN on each*

| Router | G0/0 (LAN) | G0/1 | G0/2 | Router-ID |
|---|---|---|---|---|
| R1 | 192.168.1.1/24 | 10.0.12.1/30 (to R2) | 10.0.13.1/30 (to R3) | 1.1.1.1 |
| R2 | 192.168.2.1/24 | 10.0.12.2/30 (to R1) | 10.0.23.1/30 (to R3) | 2.2.2.2 |
| R3 | 192.168.3.1/24 | 10.0.13.2/30 (to R1) | 10.0.23.2/30 (to R2) | 3.3.3.3 |

PCs: PC1 192.168.1.10, PC2 192.168.2.10, PC3 192.168.3.10, mask 255.255.255.0, gateway = .1 of their LAN.

### Dijkstra worked by hand (root R1, every link cost 1)

| Iteration | Permanent set | R2 | R3 | LAN1 | LAN2 | LAN3 |
|---|---|---|---|---|---|---|
| 0 | {R1} | 1 (R1) | 1 (R1) | 1 (direct) | ∞ | ∞ |
| 1 | {R1, R2} | 1 ✔ | 1 | 1 | 2 (via R2) | ∞ |
| 2 | {R1, R2, R3} | 1 | 1 ✔ | 1 | 2 | 2 (via R3) |
| 3 | all | | | | 2 ✔ | 2 ✔ |

So R1 should reach 192.168.2.0 via R2 and 192.168.3.0 via R3, each with total cost 2 – which is exactly what `show ip route` shows in Step 6.

## Procedure

### Step 1. Place the devices

Open Cisco Packet Tracer → *File* → *New*. From the device-type selection box at the bottom-left drag the following onto the Logical workspace (see the “Packet Tracer basics” section, Figure B.1):

- 3 × **2911** routers (R1, R2, R3)
- 3 × PCs (PC1, PC2, PC3)

Click each device and change its **Display Name** (Config tab → Global → Settings) to match the diagram, so the labels in your screenshots match the procedure.

### Step 2. Connect the cables

Click the **Connections** (lightning-bolt) icon. Choose the cable type, click the first device, pick the port from the pop-up list, then click the second device and pick its port:

- Copper straight-through (or cross-over): R1 G0/1 ↔ R2 G0/1, R1 G0/2 ↔ R3 G0/1, R2 G0/2 ↔ R3 G0/2
- Copper **cross-over**: each PC FastEthernet0 ↔ its router G0/0 (a PC connects directly to a router port; a switch can be inserted instead)

Wait until the link lights turn **green** (switch ports take about 30 s to move from amber to green). Router ports stay **red** until you enter `no shutdown`.

### Step 3. Configure interfaces and OSPF on R1

```
Router> enable
Router# configure terminal
Router(config)# hostname R1
R1(config)# interface g0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# interface g0/1
R1(config-if)# ip address 10.0.12.1 255.255.255.252
R1(config-if)# no shutdown
R1(config-if)# interface g0/2
R1(config-if)# ip address 10.0.13.1 255.255.255.252
R1(config-if)# no shutdown
R1(config-if)# exit
R1(config)# router ospf 1
R1(config-router)# router-id 1.1.1.1
R1(config-router)# network 192.168.1.0 0.0.0.255 area 0
R1(config-router)# network 10.0.12.0 0.0.0.3 area 0
R1(config-router)# network 10.0.13.0 0.0.0.3 area 0
R1(config-router)# end
R1# write memory
```

### Step 4. Configure R2 and R3

R2:

```
hostname R2
interface g0/0
 ip address 192.168.2.1 255.255.255.0
 no shutdown
interface g0/1
 ip address 10.0.12.2 255.255.255.252
 no shutdown
interface g0/2
 ip address 10.0.23.1 255.255.255.252
 no shutdown
router ospf 1
 router-id 2.2.2.2
 network 192.168.2.0 0.0.0.255 area 0
 network 10.0.12.0 0.0.0.3 area 0
 network 10.0.23.0 0.0.0.3 area 0
```

R3:

```
hostname R3
interface g0/0
 ip address 192.168.3.1 255.255.255.0
 no shutdown
interface g0/1
 ip address 10.0.13.2 255.255.255.252
 no shutdown
interface g0/2
 ip address 10.0.23.2 255.255.255.252
 no shutdown
router ospf 1
 router-id 3.3.3.3
 network 192.168.3.0 0.0.0.255 area 0
 network 10.0.13.0 0.0.0.3 area 0
 network 10.0.23.0 0.0.0.3 area 0
```

(Type these after `enable` and `configure terminal`; each `interface` line moves you into that interface automatically.)

### Step 5. Configure the end devices (static IP)

Click a PC → **Desktop** tab → **IP Configuration** → select **Static** and type the values from the table (Figure B.2). For a server use *Config* → *FastEthernet0* for the address and *Config* → *Global* → *Settings* for the gateway and DNS.

| Device | IPv4 Address | Subnet Mask | Default Gateway | DNS Server |
|---|---|---|---|---|
| PC1 | 192.168.1.10 | 255.255.255.0 | 192.168.1.1 | – |
| PC2 | 192.168.2.10 | 255.255.255.0 | 192.168.2.1 | – |
| PC3 | 192.168.3.10 | 255.255.255.0 | 192.168.3.1 | – |

### Step 6. Verify neighbours, the link-state database and the routes

On R1:

```
R1# show ip ospf neighbor
Neighbor ID     Pri   State           Dead Time   Address         Interface
2.2.2.2           1   FULL/DR         00:00:33    10.0.12.2       GigabitEthernet0/1
3.3.3.3           1   FULL/DR         00:00:38    10.0.13.2       GigabitEthernet0/2

R1# show ip ospf database
            OSPF Router with ID (1.1.1.1) (Process ID 1)
                Router Link States (Area 0)
Link ID         ADV Router      Age    Seq#       Checksum Link count
1.1.1.1         1.1.1.1         ...    0x8000000x  ...      3
2.2.2.2         2.2.2.2         ...    0x8000000x  ...      3
3.3.3.3         3.3.3.3         ...    0x8000000x  ...      3
                Net Link States (Area 0)   ...

R1# show ip route ospf
O    192.168.2.0/24 [110/2] via 10.0.12.2, 00:02:10, GigabitEthernet0/1
O    192.168.3.0/24 [110/2] via 10.0.13.2, 00:02:10, GigabitEthernet0/2
     10.0.0.0/8 is variably subnetted ...
```

```
O       10.0.23.0/30 [110/2] via 10.0.12.2, 00:02:10, GigabitEthernet0/1
                     [110/2] via 10.0.13.2, 00:02:10, GigabitEthernet0/2
```

All three routers appear in every router's LSDB (the same map everywhere). The 10.0.23.0 network has two equal-cost paths, so OSPF installs both (load balancing). Exact ages, sequence numbers and DR/BDR roles in your output will differ.

### Step 7. Test end-to-end connectivity

PC1 → Command Prompt:

```
C:\> ping 192.168.3.10
Reply from 192.168.3.10: bytes=32 time<1ms TTL=126   (x4 after the first ARP)

C:\> tracert 192.168.3.10
  1   0 ms   0 ms   0 ms   192.168.1.1
  2   0 ms   0 ms   0 ms   10.0.13.2
  3   0 ms   0 ms   0 ms   192.168.3.10
```

The direct R1 → R3 link is used (shortest path).

### Step 8. Link failure – watch OSPF re-converge

On R1: `configure terminal` → `interface g0/2` → `shutdown`. A message `%OSPF-5-ADJCHG ... 3.3.3.3 ... from FULL to DOWN` appears. Run `show ip route ospf` again:

```
O    192.168.3.0/24 [110/3] via 10.0.12.2, 00:00:04, GigabitEthernet0/1
```

The cost rose to 3 and the next hop is now R2. `tracert 192.168.3.10` from PC1 shows 192.168.1.1 → 10.0.12.2 → 10.0.23.2 → 192.168.3.10. Re-enable the link with `no shutdown`.

### Step 9. Change the link cost (metric-based path selection)

On R1 and R3 set a high cost on the direct link: `interface g0/2` (R1) / `interface g0/1` (R3) → `ip ospf cost 10`. Now the path R1 → R2 → R3 (cost 1 + 1 + 1 = 3) is cheaper than the direct link (10 + 1 = 11), so R1's route to 192.168.3.0 changes to [110/3] via 10.0.12.2 even though the direct link is up. Remove with `no ip ospf cost`.

### Step 10. See the LSAs in Simulation mode

Simulation → Edit Filters → OSPF only. Shut and re-enable a link and press Capture/Forward to see the OSPF LSU (Link-State Update) packets flooding to all routers, and the Hello packets every 10 s.

*Screenshot – show ip ospf neighbor and show ip route on R1*

(paste your own screenshot here, with your ID card visible)

*Screenshot – tracert before and after shutting the R1–R3 link*

(paste your own screenshot here, with your ID card visible)

## Result

OSPF (link-state routing) was configured on three routers in area 0. Each router formed FULL adjacencies with its two neighbours, held an identical link-state database, and installed the least-cost paths computed by Dijkstra's algorithm (cost 2 to each remote LAN). When the R1–R3 link was shut down or its cost raised, the routers recomputed the shortest path and traffic moved through R2 with cost 3, without losing connectivity.

## Conclusion

Link-state routing gives every router a full map of the network and uses Dijkstra's SPF algorithm to select least-cost paths, so it converges quickly, chooses routes by link cost rather than hop count and reacts immediately to topology changes.

EXPERIMENT
12

# Implement Unicast Routing with the help of Distance Vector Routing (DVR) using CPT

*Tool: Cisco Packet Tracer • Protocol: RIP v2*

Date: _____ Name: _____ Reg. No: _____

## Aim

To implement distance-vector unicast routing using RIP version 2 on a chain of three routers in Cisco Packet Tracer, observe how routes and hop counts are learned from neighbours, and see the periodic updates and convergence after a link failure.

## Theory

In **distance-vector routing** each router knows only the distance (metric) and direction (next hop) to each destination. Periodically it sends its whole *distance vector* to its neighbours, and each router applies the **Bellman-Ford** equation:

$$D_x(y) = \min_{\text{over neighbours } v} \{ c(x,v) + D_v(y) \}$$

RIP uses hop count as the metric (each router hop = 1, 16 = infinity), sends updates every 30 s, and marks a route invalid after 180 s without updates. To limit the **count-to-infinity** problem it uses **split horizon** (do not advertise a route back out of the interface it was learned on), **route poisoning** (advertise a failed route with metric 16) and hold-down timers.

## Network topology

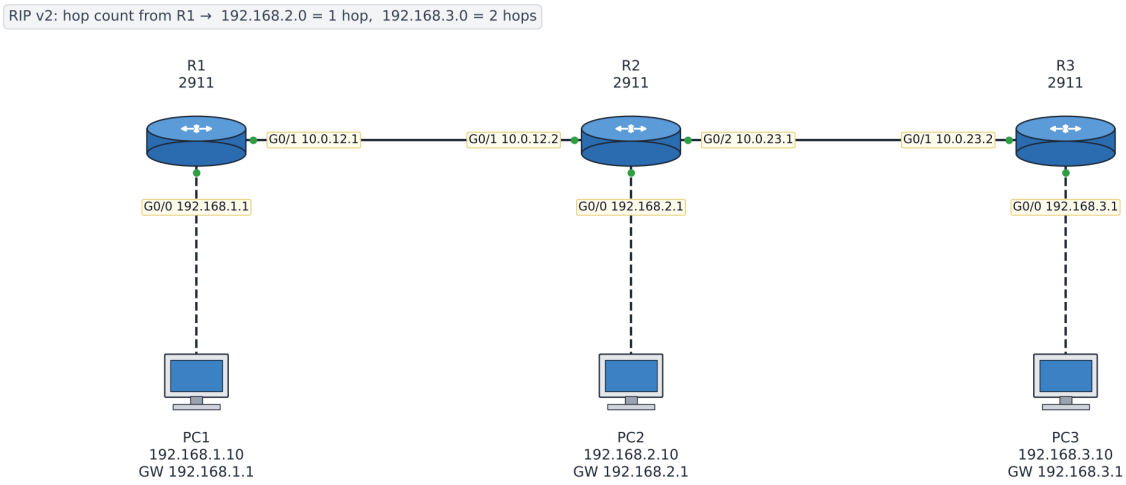



*Figure 12.1 – Three routers in a chain*

| Router | G0/0 (LAN) | G0/1 | G0/2 |
|---|---|---|---|
| R1 | 192.168.1.1/24 | 10.0.12.1/30 (to R2) | – |
| R2 | 192.168.2.1/24 | 10.0.12.2/30 (to R1) | 10.0.23.1/30 (to R3) |
| R3 | 192.168.3.1/24 | 10.0.23.2/30 (to R2) | – |

### How R1's distance vector converges (Bellman-Ford)

| Destination | Start (t = 0) | After 1st update | After 2nd update |
|---|---|---|---|
| 192.168.1.0 | 0 – connected | 0 – connected | 0 – connected |
| 10.0.12.0 | 0 – connected | 0 – connected | 0 – connected |
| 192.168.2.0 | ∞ | 1 via R2 | 1 via R2 |

| Destination | Start (t = 0) | After 1st update | After 2nd update |
|---|---|---|---|
| 10.0.23.0 | ∞ | 1 via R2 | 1 via R2 |
| 192.168.3.0 | ∞ | ∞ (R2 has not learned it yet) | 2 via R2 |

## Procedure

### Step 1. Place the devices

Open Cisco Packet Tracer → *File* → *New*. From the device-type selection box at the bottom-left drag the following onto the Logical workspace (see the “Packet Tracer basics” section, Figure B.1):

- 3 × **2911** routers (R1, R2, R3)
- 3 × PCs (PC1, PC2, PC3)

Click each device and change its **Display Name** (Config tab → Global → Settings) to match the diagram, so the labels in your screenshots match the procedure.

### Step 2. Connect the cables

Click the **Connections** (lightning-bolt) icon. Choose the cable type, click the first device, pick the port from the pop-up list, then click the second device and pick its port:

- Copper straight-through: R1 G0/1 ↔ R2 G0/1 and R2 G0/2 ↔ R3 G0/1
- Copper cross-over: PC1 ↔ R1 G0/0, PC2 ↔ R2 G0/0, PC3 ↔ R3 G0/0

Wait until the link lights turn **green** (switch ports take about 30 s to move from amber to green). Router ports stay **red** until you enter `no shutdown`.

### Step 3. Configure R1

```
Router> enable
Router# configure terminal
Router(config)# hostname R1
R1(config)# interface g0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# interface g0/1
R1(config-if)# ip address 10.0.12.1 255.255.255.252
R1(config-if)# no shutdown
R1(config-if)# exit
R1(config)# router rip
R1(config-router)# version 2
R1(config-router)# no auto-summary
R1(config-router)# network 192.168.1.0
R1(config-router)# network 10.0.0.0
R1(config-router)# end
```

### Step 4. Configure R2 and R3

R2:

```
hostname R2
interface g0/0
 ip address 192.168.2.1 255.255.255.0
 no shutdown
interface g0/1
 ip address 10.0.12.2 255.255.255.252
 no shutdown
interface g0/2
 ip address 10.0.23.1 255.255.255.252
 no shutdown
router rip
 version 2
 no auto-summary
 network 192.168.2.0
 network 10.0.0.0
```

R3:

```
hostname R3
interface g0/0
 ip address 192.168.3.1 255.255.255.0
 no shutdown
```

```
interface g0/1
 ip address 10.0.23.2 255.255.255.252
 no shutdown
router rip
 version 2
 no auto-summary
 network 192.168.3.0
 network 10.0.0.0
```

RIP network statements are classful, so `network 10.0.0.0` enables RIP on every 10.x.x.x interface. `no auto-summary` stops RIP from summarising 10.0.12.0/30 and 10.0.23.0/30 into 10.0.0.0/8.

### Step 5. Configure the end devices (static IP)

Click a PC → **Desktop** tab → **IP Configuration** → select **Static** and type the values from the table (Figure B.2). For a server use *Config* → *FastEthernet0* for the address and *Config* → *Global* → *Settings* for the gateway and DNS.

| Device | IPv4 Address | Subnet Mask | Default Gateway | DNS Server |
|---|---|---|---|---|
| PC1 | 192.168.1.10 | 255.255.255.0 | 192.168.1.1 | – |
| PC2 | 192.168.2.10 | 255.255.255.0 | 192.168.2.1 | – |
| PC3 | 192.168.3.10 | 255.255.255.0 | 192.168.3.1 | – |

### Step 6. Watch the distance vectors being exchanged

Immediately after configuring, run `debug ip rip` on R2:

```
RIP: sending  v2 update to 224.0.0.9 via GigabitEthernet0/1 (10.0.12.2)
RIP: build update entries
      10.0.23.0/30 via 0.0.0.0, metric 1, tag 0
      192.168.2.0/24 via 0.0.0.0, metric 1, tag 0
      192.168.3.0/24 via 0.0.0.0, metric 2, tag 0
RIP: received v2 update from 10.0.12.1 on GigabitEthernet0/1
      192.168.1.0/24 via 0.0.0.0 in 1 hops
```

Note split horizon: R2 does not send 192.168.1.0 back to R1. Stop with `undebug all`.

### Step 7. Verify the routing tables

On R1 (after about 30–60 s):

```
R1# show ip route rip
     10.0.0.0/8 is variably subnetted, 3 subnets, 2 masks
R       10.0.23.0/30 [120/1] via 10.0.12.2, 00:00:21, GigabitEthernet0/1
R    192.168.2.0/24 [120/1] via 10.0.12.2, 00:00:21, GigabitEthernet0/1
R    192.168.3.0/24 [120/2] via 10.0.12.2, 00:00:21, GigabitEthernet0/1

R1# show ip protocols
Routing Protocol is "rip"
Sending updates every 30 seconds, next due in 12 seconds
Invalid after 180 seconds, hold down 180, flushed after 240
...
Default version control: send version 2, receive 2
```

The metric in brackets is the hop count: 1 to R2's LAN, 2 to R3's LAN, matching the Bellman-Ford table.

### Step 8. Test connectivity

```
PC1> ping 192.168.3.10      -> replies, TTL=125 (three routers)
PC1> tracert 192.168.3.10
  1   192.168.1.1
  2   10.0.12.2
  3   10.0.23.2
  4   192.168.3.10
```

### Step 9. Link failure and route poisoning

On R2 shut G0/2 (`interface g0/2 → shutdown`). With `debug ip rip` on R1 you will see 192.168.3.0 advertised with metric 16 (unreachable) and it disappears from R1's table; ping PC1 →

PC3 fails. Bring the link back with `no shutdown` and after the next update (≤ 30 s) the route returns with metric 2.

### Step 10. Observe RIP packets

Simulation mode → Edit Filters → RIP only. Press Play: a RIPv2 update leaves every router interface every 30 s. Click one and open the PDU details to see the UDP port 520 and the list of routes and metrics it carries.

*Screenshot – show ip route on R1 with [120/1] and [120/2] routes*

(paste your own screenshot here, with your ID card visible)

*Screenshot – ping / tracert from PC1 to PC3*

(paste your own screenshot here, with your ID card visible)

## Result

RIP v2 was configured on R1, R2 and R3. Each router learned the remote networks from its neighbours with the correct hop counts (R1: 192.168.2.0 at 1 hop, 192.168.3.0 at 2 hops), PC1 reached PC3 across three routers, and a link failure was propagated as an unreachable (metric 16) route and recovered after the link came back.

## Conclusion

Distance-vector routing builds routes by repeatedly exchanging distance vectors with neighbours (Bellman-Ford). It is simple to configure but converges slowly, uses only hop count, and needs split horizon and route poisoning to avoid count-to-infinity loops.

EXPERIMENT 13

# Implement Unicast Routing with the help of BGP using CPT

*Tool: Cisco Packet Tracer • Protocol: eBGP*

Date: _____ Name: _____ Reg. No: _____

## Aim

To implement inter-domain unicast routing using the Border Gateway Protocol (BGP) between three autonomous systems in Cisco Packet Tracer and verify BGP neighbours, the BGP table (AS path) and end-to-end connectivity.

## Theory

The Internet is made of **Autonomous Systems (AS)** – networks under one administration, each with an AS number. Inside an AS an IGP such as OSPF or RIP is used; **between** ASes the routing protocol is **BGP-4**, a *path-vector* protocol.

- BGP neighbours (peers) are configured manually and talk over **TCP port 179**.
- **eBGP** runs between routers in different ASes (administrative distance 20); iBGP runs inside one AS (distance 200).
- Each route carries the **AS_PATH** attribute – the list of ASes it has crossed. A router rejects a route containing its own AS number, which prevents loops, and normally prefers the shortest AS path.
- BGP sends full routes once and then only incremental updates; KEEPALIVE messages every 60 s keep the session up.
- Neighbour states: Idle → Connect → Active → OpenSent → OpenConfirm → **Established**.

## Network topology

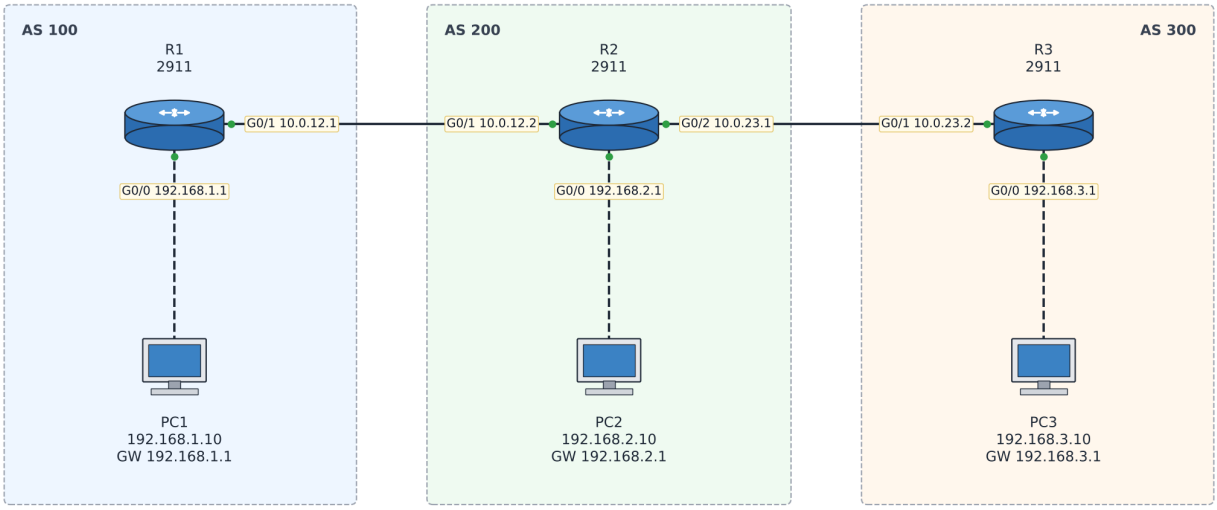



*Figure 13.1 – Three autonomous systems AS 100, AS 200, AS 300*

| Router | AS | G0/0 (LAN) | G0/1 | G0/2 | BGP neighbours |
|---|---|---|---|---|---|
| R1 | 100 | 192.168.1.1/24 | 10.0.12.1/30 | – | 10.0.12.2 (AS 200) |
| R2 | 200 | 192.168.2.1/24 | 10.0.12.2/30 | 10.0.23.1/30 | 10.0.12.1 (AS 100), 10.0.23.2 (AS 300) |
| R3 | 300 | 192.168.3.1/24 | 10.0.23.2/30 | – | 10.0.23.1 (AS 200) |

## Procedure

### Step 1. Place the devices

Open Cisco Packet Tracer → *File* → *New*. From the device-type selection box at the bottom-left drag the following onto the Logical workspace (see the "Packet Tracer basics" section, Figure B.1):

- 3 × **2911** routers (R1, R2, R3)
- 3 × PCs

Click each device and change its **Display Name** (Config tab → Global → Settings) to match the diagram, so the labels in your screenshots match the procedure.

### Step 2. Connect the cables

Click the **Connections** (lightning-bolt) icon. Choose the cable type, click the first device, pick the port from the pop-up list, then click the second device and pick its port:

- Copper straight-through: R1 G0/1 ↔ R2 G0/1, R2 G0/2 ↔ R3 G0/1
- Copper cross-over: PC1 ↔ R1 G0/0, PC2 ↔ R2 G0/0, PC3 ↔ R3 G0/0

Wait until the link lights turn **green** (switch ports take about 30 s to move from amber to green). Router ports stay **red** until you enter `no shutdown`.

### Step 3. Configure the interfaces

Configure the IP addresses from the table exactly as in Experiment 12, Steps 3–4, but **without** any `router rip` section (no IGP is needed because the eBGP peers are directly connected). Check with `show ip interface brief` and ping each neighbour's address (e.g. from R1: `ping 10.0.12.2`).

### Step 4. Configure BGP on R1 (AS 100)

```
R1(config)# router bgp 100
R1(config-router)# neighbor 10.0.12.2 remote-as 200
R1(config-router)# network 192.168.1.0 mask 255.255.255.0
R1(config-router)# end
```

### Step 5. Configure BGP on R2 (AS 200)

```
R2(config)# router bgp 200
R2(config-router)# neighbor 10.0.12.1 remote-as 100
R2(config-router)# neighbor 10.0.23.2 remote-as 300
R2(config-router)# network 192.168.2.0 mask 255.255.255.0
R2(config-router)# end
```

### Step 6. Configure BGP on R3 (AS 300)

```
R3(config)# router bgp 300
R3(config-router)# neighbor 10.0.23.1 remote-as 200
R3(config-router)# network 192.168.3.0 mask 255.255.255.0
R3(config-router)# end
```

Within a few seconds each router prints `%BGP-5-ADJCHANGE: neighbor 10.0.x.x Up`. The `network ... mask ...` statement must match a route already in the routing table exactly (here the connected LAN), otherwise it is not advertised.

### Step 7. Configure the end devices (static IP)

Click a PC → **Desktop** tab → **IP Configuration** → select **Static** and type the values from the table (Figure B.2). For a server use *Config* → *FastEthernet0* for the address and *Config* → *Global* → *Settings* for the gateway and DNS.

| Device | IPv4 Address | Subnet Mask | Default Gateway | DNS Server |
|---|---|---|---|---|
| PC1 | 192.168.1.10 | 255.255.255.0 | 192.168.1.1 | – |
| PC2 | 192.168.2.10 | 255.255.255.0 | 192.168.2.1 | – |
| PC3 | 192.168.3.10 | 255.255.255.0 | 192.168.3.1 | – |

### Step 8. Verify the BGP session and BGP table

On R1:

```
R1# show ip bgp summary
```

```
BGP router identifier 192.168.1.1, local AS number 100
...
Neighbor        V    AS MsgRcvd MsgSent   TblVer  InQ OutQ Up/Down  State/PfxRcd
10.0.12.2       4   200       8       6        4    0    0 00:03:10        2

R1# show ip bgp
   Network          Next Hop            Metric LocPrf Weight Path
*> 192.168.1.0/24   0.0.0.0                  0         32768 i
*> 192.168.2.0/24   10.0.12.2                0             0 200 i
*> 192.168.3.0/24   10.0.12.2                0             0 200 300 i

R1# show ip route bgp
B    192.168.2.0/24 [20/0] via 10.0.12.2, 00:03:05
B    192.168.3.0/24 [20/0] via 10.0.12.2, 00:03:05
```

**State/PfxRcd = 2** means the session is Established and 2 prefixes were received. The **Path** column is the AS_PATH: R3's LAN is reached through AS 200 then AS 300. Routes are marked **B** with administrative distance 20 (eBGP). Run the same commands on R2 and R3.

### Step 9. Test end-to-end connectivity

```
PC1> ping 192.168.3.10       -> Reply ... TTL=125
PC1> tracert 192.168.3.10
  1   192.168.1.1
  2   10.0.12.2
  3   10.0.23.2
  4   192.168.3.10
```

### Step 10. Observe BGP messages

Simulation mode → Edit Filters → BGP (and TCP). On R2 run `clear ip bgp *` and press Capture/Forward: you see the TCP handshake to port 179, then BGP OPEN, KEEPALIVE and UPDATE messages carrying the prefixes and AS_PATH.

**Troubleshooting:** Packet Tracer supports a subset of BGP (basic eBGP neighbours and network statements). If `show ip bgp summary` shows the neighbour in Idle/Active, check the neighbour address and remote-as number on both sides and that the link can be pinged.

*Screenshot – show ip bgp summary / show ip bgp on R1 with AS path*

(paste your own screenshot here, with your ID card visible)

*Screenshot – ping from PC1 to PC3*

(paste your own screenshot here, with your ID card visible)

## Result

eBGP sessions were established between AS 100–AS 200 and AS 200–AS 300. Each router advertised its LAN, and R1 learned 192.168.2.0/24 with AS path 200 and 192.168.3.0/24 with AS path 200 300, installed as B routes with distance 20. PC1 in AS 100 successfully reached PC3 in AS 300 through AS 200.

## Conclusion

BGP is the path-vector protocol that routes between autonomous systems: peers are configured explicitly over TCP 179, and the AS_PATH attribute both describes the route and prevents loops, which makes BGP suitable for policy-based routing across the Internet.

EXPERIMENT
14

# Perform Traffic Control Operations using TCP and UDP with the help of CPT

*Tool: Cisco Packet Tracer*

Date: _____ Name: _____ Reg. No: _____

## Aim

To generate TCP (HTTP) and UDP (DNS) traffic in Cisco Packet Tracer, analyse the TCP segment and UDP datagram headers in simulation mode, observe the TCP connection set-up and teardown and multiple simultaneous flows, and control traffic by filtering specific TCP and UDP ports with an extended access control list (ACL).

## Theory

| Feature | TCP | UDP |
|---|---|---|
| Connection | Connection-oriented: 3-way handshake (SYN, SYN-ACK, ACK) and FIN teardown | Connectionless: no handshake |
| Reliability | Sequence and acknowledgement numbers, retransmission | None (best effort) |
| Flow control | Sliding window (Window field) | None |
| Congestion control | Slow start, congestion avoidance | None |
| Header size | 20–60 bytes | 8 bytes (src port, dst port, length, checksum) |
| Examples | HTTP 80, HTTPS 443, FTP 21, SMTP 25 | DNS 53, DHCP 67/68, TFTP 69, RIP 520 |

**Traffic control** on a router is done with **extended ACLs**, which match on protocol (tcp/udp/icmp/ip), source, destination and port numbers. Rules are checked top-down; the first match wins, and every ACL ends with an invisible *deny any*, so a final *permit ip any any* is required to allow other traffic.

## Network topology

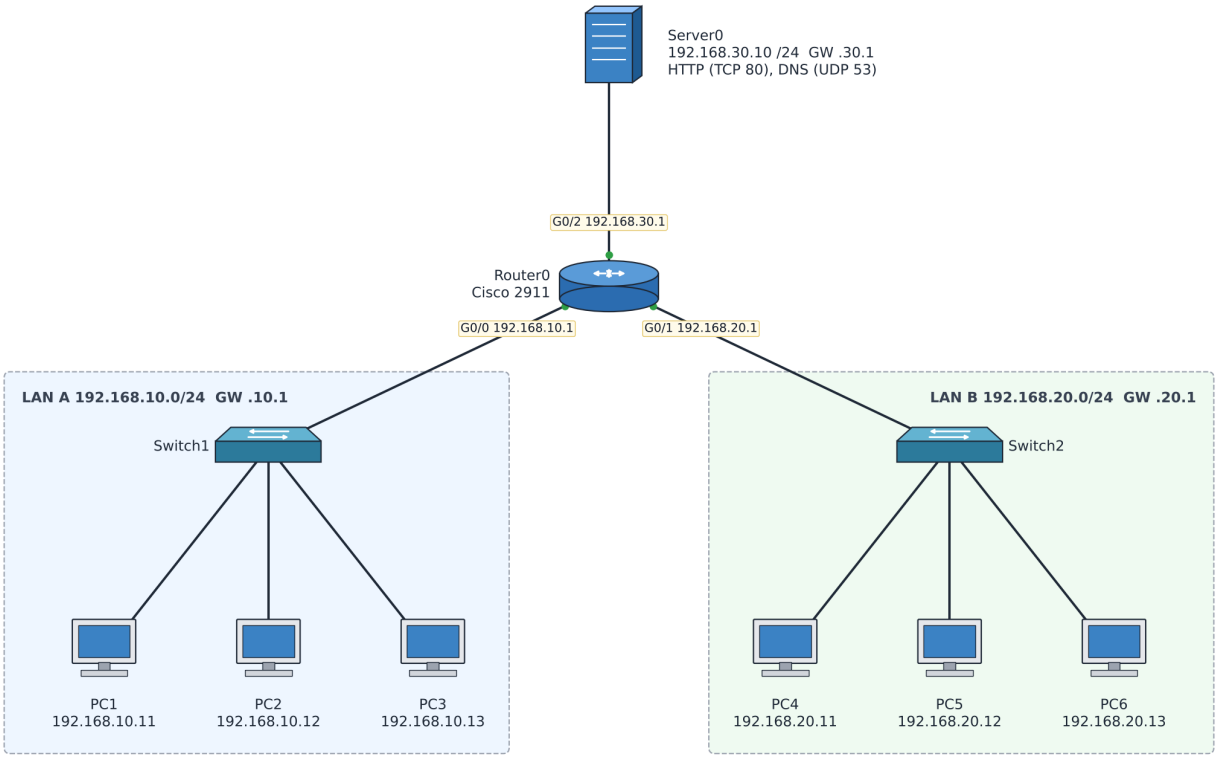



*Figure 14.1 – Two client LANs and a server LAN behind one router*

| Device | Interface | IP address / mask | Gateway | DNS |
|---|---|---|---|---|
| Router0 | G0/0 / G0/1 / G0/2 | `192.168.10.1 / 192.168.20.1 / 192.168.30.1 (/24)` | – | – |
| Server0 | Fa0 | `192.168.30.10 /24` | `192.168.30.1` | `192.168.30.10` |
| PC1 – PC3 | Fa0 | `192.168.10.11 – .13 /24` | `192.168.10.1` | `192.168.30.10` |
| PC4 – PC6 | Fa0 | `192.168.20.11 – .13 /24` | `192.168.20.1` | `192.168.30.10` |

# Procedure

## Part A – Build and test the network

### Step 1. Place the devices

Open Cisco Packet Tracer → *File* → *New*. From the device-type selection box at the bottom-left drag the following onto the Logical workspace (see the “Packet Tracer basics” section, Figure B.1):

- 1 × 2911 router, 2 × 2960 switches, 1 × Server, 6 × PCs

Click each device and change its **Display Name** (Config tab → Global → Settings) to match the diagram, so the labels in your screenshots match the procedure.

### Step 2. Connect the cables

Click the **Connections** (lightning-bolt) icon. Choose the cable type, click the first device, pick the port from the pop-up list, then click the second device and pick its port:

- Router0 G0/0 → Switch1, G0/1 → Switch2, G0/2 → Server0 (straight-through)
- PC1–PC3 → Switch1, PC4–PC6 → Switch2

Wait until the link lights turn **green** (switch ports take about 30 s to move from amber to green). Router ports stay **red** until you enter `no shutdown`.

### Step 3. Configure the router

```
Router(config)# hostname R0
R0(config)# interface g0/0
R0(config-if)# ip address 192.168.10.1 255.255.255.0
R0(config-if)# no shutdown
```

```
R0(config-if)# interface g0/1
R0(config-if)# ip address 192.168.20.1 255.255.255.0
R0(config-if)# no shutdown
R0(config-if)# interface g0/2
R0(config-if)# ip address 192.168.30.1 255.255.255.0
R0(config-if)# no shutdown
R0(config-if)# end
```

### Step 4. Configure the server and PCs

Server0: IP 192.168.30.10/24, gateway 192.168.30.1, DNS 192.168.30.10. Services → **HTTP**: On. Services → **DNS**: On, add A record `www.test.com → 192.168.30.10`. Give PCs the addresses from the table with DNS server 192.168.30.10.

### Step 5. Connectivity test

From PC1 ping 192.168.30.10 and 192.168.20.11. The first ping may lose one packet to ARP (25% loss); the second shows Sent = 4, Received = 4, Lost = 0.

## Part B – Analyse TCP

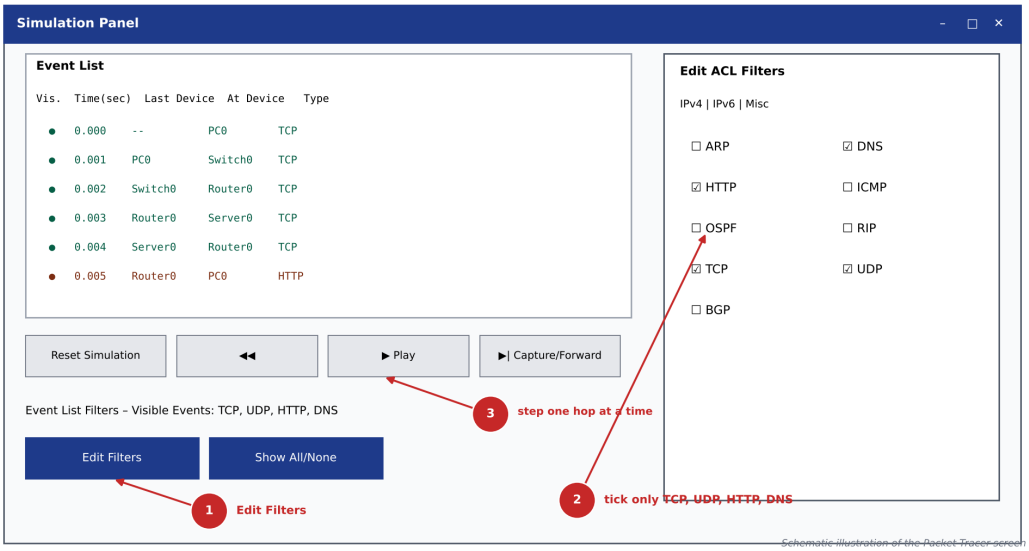



*Figure 14.2 – Simulation panel: Edit Filters and Capture/Forward*

### Step 6. Capture an HTTP (TCP) session

Click **Simulation** (bottom-right). Edit Filters → Show None, then tick **TCP** and **HTTP**. On PC1 open Desktop → **Web Browser**, type `http://192.168.30.10` and click Go. Press **Capture/Forward** repeatedly and watch the event list.

### Step 7. Read the TCP header

Click the coloured envelope of each event (at Switch1 or Router0) → **Inbound PDU Details**. Record the fields:

| Segment | Src port | Dst port | Flags | Seq / Ack | Meaning |
|---|---|---|---|---|---|
| 1 PC1 → Server | 1025* | 80 | SYN (0x02) | Seq 0 | client requests connection |
| 2 Server → PC1 | 80 | 1025* | SYN + ACK (0x12) | Seq 0 / Ack 1 | server accepts |
| 3 PC1 → Server | 1025* | 80 | ACK (0x10) | Seq 1 / Ack 1 | connection established |
| 4 PC1 → Server | 1025* | 80 | PSH + ACK | data = HTTP GET | request |
| 5 Server → PC1 | 80 | 1025* | PSH + ACK | data = HTTP 200 OK | web page |
| 6+ both | | | FIN + ACK, ACK | | connection closed |

* the client port is an ephemeral port chosen by the PC (1025, 1026 …); relative sequence numbers are shown – Packet Tracer may show actual numbers. Also note the **Window** field (receiver's buffer space used for flow control) and the **Checksum**.

## Part C – Analyse UDP

### Step 8. Capture a DNS (UDP) query

Reset Simulation. Edit Filters → tick only **DNS** and **UDP**. On PC1 → Command Prompt run `nslookup www.test.com` and press Capture/Forward. Open the PDU details:

| Field | Query (PC1 → Server) | Response (Server → PC1) |
|---|---|---|
| Source port | 1025 (ephemeral) | 53 |
| Destination port | 53 | 1025 |
| Length / Checksum | UDP header + DNS query | UDP header + DNS answer |
| Handshake before data? | No | – |
| DNS section | QDCOUNT 1, NAME www.test.com | ANCOUNT 1, A 192.168.30.10 |

Only two datagrams are exchanged: no connection set-up, no acknowledgement and no window field.

## Part D – Multiple simultaneous TCP flows

### Step 9. Generate concurrent traffic

Filter TCP only. Start web requests from PC1, PC2 and PC4 to http://192.168.30.10 one after the other without pressing Capture/Forward, then press Play. The event list shows the segments of the three sessions interleaved at Router0; each flow is kept separate by its unique (source IP, source port) pair, and each completes its own handshake independently.

## Part E – Traffic control with an extended ACL

### Step 10. Create and apply the ACL on Router0

Policy: PC2 may **not** browse the web server (TCP 80); PC3 may **not** use DNS (UDP 53); everything else is allowed.

```
R0(config)# access-list 110 deny tcp host 192.168.10.12 host 192.168.30.10 eq 80
R0(config)# access-list 110 deny udp host 192.168.10.13 host 192.168.30.10 eq 53
R0(config)# access-list 110 permit ip any any
R0(config)# interface g0/0
R0(config-if)# ip access-group 110 in
R0(config-if)# end
```

Extended ACLs are applied as close to the **source** as possible, here inbound on G0/0 (LAN A).

### Step 11. Verify the traffic control

| Test | Expected result |
|---|---|
| PC1 browser → http://192.168.30.10 | Page loads (permitted) |
| PC2 browser → http://192.168.30.10 | Request Timeout – TCP 80 blocked |
| PC2 ping 192.168.30.10 | Replies – ICMP still permitted |
| PC3 nslookup www.test.com | Fails / times out – UDP 53 blocked |
| PC3 browser → http://192.168.30.10 (by IP) | Page loads – only DNS was blocked |
| PC4 – PC6 anything | Works – ACL applies only to traffic entering G0/0 |

```
R0# show access-lists
Extended IP access list 110
    10 deny tcp host 192.168.10.12 host 192.168.30.10 eq www (4 match(es))
    20 deny udp host 192.168.10.13 host 192.168.30.10 eq domain (2 match(es))
    30 permit ip any any (27 match(es))
```

The match counters show which rule handled the traffic (your counts will differ). In Simulation mode the blocked packet is shown with a red ✕ at Router0. Remove the filter with `no ip access-group 110 in`.

*Screenshot – TCP PDU details (ports, seq, ack, flags, window)*

(paste your own screenshot here, with your ID card visible)

*Screenshot – UDP / DNS PDU details*

(paste your own screenshot here, with your ID card visible)

*Screenshot – ACL test: blocked browser on PC2 and show access-lists*

(paste your own screenshot here, with your ID card visible)

## Result

TCP and UDP traffic were generated and analysed in simulation mode. HTTP used TCP from an ephemeral port to port 80 with a SYN, SYN-ACK, ACK handshake, sequence/acknowledgement numbers and a window field; DNS used a single UDP datagram from an ephemeral port to port 53 and one reply. Several TCP flows were handled simultaneously. An extended ACL on the router successfully blocked HTTP from PC2 and DNS from PC3 while permitting all other traffic.

## Conclusion

TCP provides reliable, connection-oriented, flow-controlled delivery at the cost of extra header fields and handshakes, while UDP is lightweight and connectionless. Because both identify applications by port numbers, routers can control traffic per application using extended ACLs that match TCP and UDP ports.

EXPERIMENT 15

# Develop a DNS Client-Server to Resolve the given Host Name (or) IP Addresses

*Language: C (UDP sockets) + Tool: Cisco Packet Tracer*

Date: _____ Name: _____ Reg. No: _____

## Aim

To develop a DNS client and server in C using UDP sockets that resolves a given host name to its IP address (forward lookup) or a given IP address to its host name (reverse lookup), and to demonstrate the same client-server name resolution in Cisco Packet Tracer.

## Theory

The **Domain Name System** is a distributed client-server database that maps names to addresses. A client (resolver) sends a query to a DNS server, normally over **UDP port 53**; the server answers from its own zone (an *authoritative* answer) or by asking other servers (non-authoritative). Common record types: **A** (name → IPv4), **AAAA** (name → IPv6), **PTR** (IP → name, used for reverse lookup in the in-addr.arpa zone), **CNAME** (alias), **MX** (mail server) and **NS** (name server).

A socket program uses: `socket()` to create a UDP socket, `bind()` (server only) to attach it to a port, `sendto()` / `recvfrom()` to exchange datagrams, and `close()`. The library functions `getaddrinfo()` and `getnameinfo()` perform forward and reverse lookups through the operating system's resolver.

## Part A – DNS client and server in C (UDP sockets)

### Design

- The **server** opens a UDP socket on port 8053 (a non-privileged stand-in for DNS port 53), waits for a query string, and answers it from its local zone table – exactly like the A records added on a DNS server. If the name is not in the table it asks the operating system's resolver (`getaddrinfo`); if that also fails it replies NXDOMAIN.
- If the query is a dotted IPv4 address the server performs a **reverse lookup** (IP → name); otherwise a **forward lookup** (name → IP).
- The **client** reads host names or IP addresses from the keyboard, sends each to the server with `sendto()`, waits up to 3 s for the reply with `recvfrom()` and prints it. Typing **exit** ends the client.
- UDP is used because, like real DNS, each query fits in a single datagram and needs no connection setup.

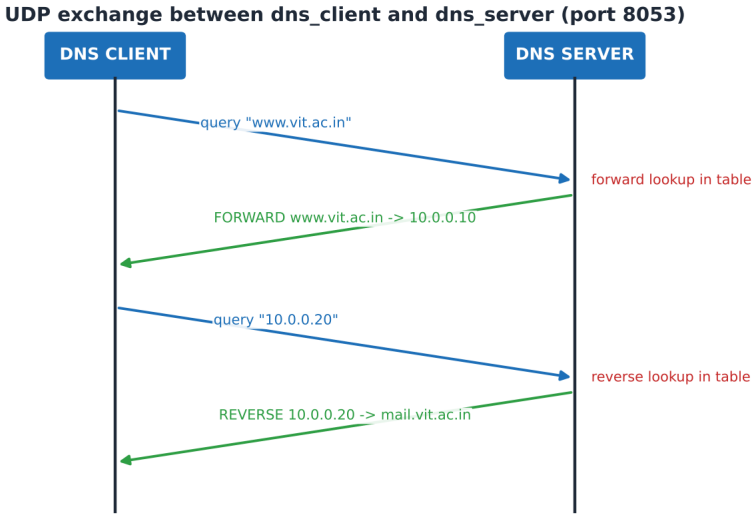



*Figure 15.1 – Query/response exchange between the client and server*

### Algorithm – server

1. Start. Create a UDP socket and bind it to INADDR_ANY, port 8053.
2. Wait for a datagram with recvfrom(); note the client's address.
3. If the query parses as an IPv4 address: search the table by IP; if not found try getnameinfo().
4. Otherwise: search the table by name (case-insensitive); if not found try getaddrinfo().
5. Send the answer (or NXDOMAIN) back to the client with sendto(); log the query.
6. Go back to step 2.

### Algorithm – client

1. Start. Read the server IP and port from the command line (default 127.0.0.1:8053).
2. Create a UDP socket and set a 3-second receive timeout.
3. Read a host name or IP address; if it is "exit", close the socket and stop.
4. Send it to the server, wait for the reply and print it (or print a timeout message).
5. Go back to step 3.

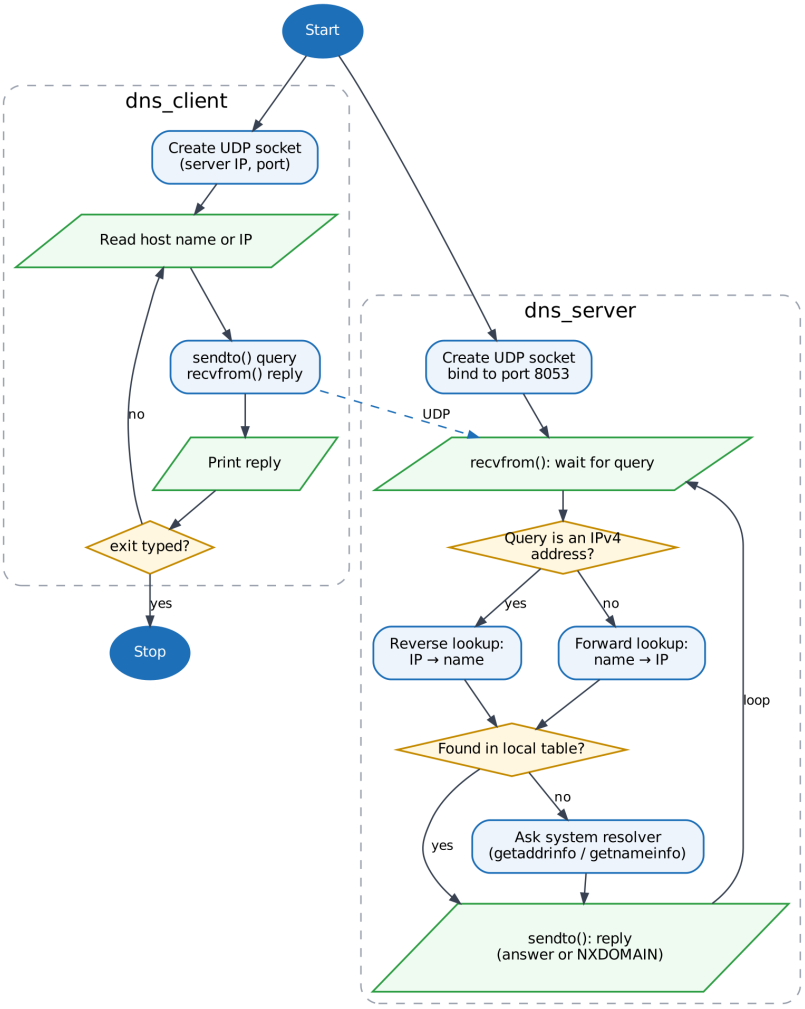



*Figure 15.2 – Flowchart of the DNS server and client*

### Program – dns_server.c

```
/* Experiment 15: DNS server (UDP) - resolves a host name to an IP address
   (forward lookup) or an IP address to a host name (reverse lookup).
   Compile: gcc dns_server.c -o dns_server      Run: ./dns_server [port] */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#ifdef _WIN32                       /* Windows (Code::Blocks / MinGW): link with -lws2_32 */
#define _WIN32_WINNT 0x0600
#include <winsock2.h>
#include <ws2tcpip.h>
#define close closesocket
#define strcasecmp _stricmp
typedef int socklen_t;
#else                               /* Linux / macOS */
```

```
#include <unistd.h>
#include <netdb.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <strings.h>
#endif

#define BUF 512

struct record { const char *name; const char *ip; };

/* the server's local zone (like the DNS records added in Packet Tracer) */
struct record table[] = {
    {"www.vit.ac.in",      "10.0.0.10"},
    {"mail.vit.ac.in",     "10.0.0.20"},
    {"ftp.vit.ac.in",      "10.0.0.30"},
    {"myweb.local",        "10.0.0.1"},
    {"dns.local",          "10.0.0.2"},
    {"localhost",          "127.0.0.1"},
};
int nrec = sizeof(table) / sizeof(table[0]);

void resolve(const char *q, char *reply)
{
    struct in_addr a;

    if (inet_pton(AF_INET, q, &a) == 1) {               /* ---- reverse lookup ---- */
        for (int i = 0; i < nrec; i++)
            if (strcmp(table[i].ip, q) == 0) {
                sprintf(reply, "REVERSE %s -> %s (authoritative, local table)", q, table[i].name);
                return;
            }
        struct sockaddr_in sa = {0};
        char host[NI_MAXHOST];
        sa.sin_family = AF_INET;
        sa.sin_addr = a;
        if (getnameinfo((struct sockaddr *)&sa, sizeof sa, host, sizeof host,
                        NULL, 0, NI_NAMEREQD) == 0)
            sprintf(reply, "REVERSE %s -> %s (non-authoritative, system resolver)", q, host);
        else
            sprintf(reply, "REVERSE %s -> NOT FOUND (NXDOMAIN)", q);
        return;
    }

    for (int i = 0; i < nrec; i++)                      /* ---- forward lookup ---- */
        if (strcasecmp(table[i].name, q) == 0) {
            sprintf(reply, "FORWARD %s -> %s (authoritative, local table)", q, table[i].ip);
            return;
        }
    struct addrinfo hints = {0}, *res;
    hints.ai_family = AF_INET;
    if (getaddrinfo(q, NULL, &hints, &res) == 0) {
        char ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &((struct sockaddr_in *)res->ai_addr)->sin_addr, ip, sizeof ip);
        sprintf(reply, "FORWARD %s -> %s (non-authoritative, system resolver)", q, ip);
        freeaddrinfo(res);
    } else {
        sprintf(reply, "FORWARD %s -> NOT FOUND (NXDOMAIN)", q);
    }
}

int main(int argc, char *argv[])
{
    int port = (argc > 1) ? atoi(argv[1]) : 8053;
#ifdef _WIN32
    WSADATA wsa;
    WSAStartup(MAKEWORD(2, 2), &wsa);
#endif
    int sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) { perror("socket"); return 1; }

    struct sockaddr_in srv = {0}, cli;
    srv.sin_family = AF_INET;
    srv.sin_addr.s_addr = INADDR_ANY;
    srv.sin_port = htons(port);
    if (bind(sock, (struct sockaddr *)&srv, sizeof srv) < 0) { perror("bind"); return 1; }

    printf("DNS server running on UDP port %d with %d records\n", port, nrec);
    fflush(stdout);

    char query[BUF], reply[BUF + 128];
```

```
    for (;;) {
        socklen_t len = sizeof cli;
        int n = recvfrom(sock, query, BUF - 1, 0, (struct sockaddr *)&cli, &len);
        if (n <= 0) continue;
        query[n] = '\0';
        query[strcspn(query, "\r\n")] = '\0';

        resolve(query, reply);
        printf("Query from %s:%d  \"%s\"  =>  %s\n",
               inet_ntoa(cli.sin_addr), ntohs(cli.sin_port), query, reply);
        fflush(stdout);
        sendto(sock, reply, strlen(reply), 0, (struct sockaddr *)&cli, len);
    }
    close(sock);
    return 0;
}
```

## Program – dns_client.c

```
/* Experiment 15: DNS client (UDP) - sends a host name or IP address to the
   DNS server and prints the resolved answer.
   Compile: gcc dns_client.c -o dns_client
   Run    : ./dns_client <server-ip> [port]      (type "exit" to quit) */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#ifdef _WIN32                       /* Windows (Code::Blocks / MinGW): link with -lws2_32 */
#define _WIN32_WINNT 0x0600
#include <winsock2.h>
#include <ws2tcpip.h>
#define close closesocket
#else                               /* Linux / macOS */
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/time.h>
#endif

#define BUF 640

int main(int argc, char *argv[])
{
    const char *server_ip = (argc > 1) ? argv[1] : "127.0.0.1";
    int port = (argc > 2) ? atoi(argv[2]) : 8053;

#ifdef _WIN32
    WSADATA wsa;
    WSAStartup(MAKEWORD(2, 2), &wsa);
#endif
    int sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) { perror("socket"); return 1; }

#ifdef _WIN32
    DWORD tv = 3000;                                  /* 3 s receive timeout */
    setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, (const char *)&tv, sizeof tv);
#else
    struct timeval tv = {3, 0};                       /* 3 s receive timeout */
    setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
#endif

    struct sockaddr_in srv = {0};
    srv.sin_family = AF_INET;
    srv.sin_port = htons(port);
    if (inet_pton(AF_INET, server_ip, &srv.sin_addr) != 1) {
        printf("Invalid server address\n");
        return 1;
    }

    char query[256], reply[BUF];
    printf("DNS client - server %s:%d\n", server_ip, port);
    for (;;) {
        printf("\nEnter host name or IP address (exit to quit): ");
        if (scanf("%255s", query) != 1 || strcmp(query, "exit") == 0)
            break;

        sendto(sock, query, strlen(query), 0, (struct sockaddr *)&srv, sizeof srv);
        int n = recvfrom(sock, reply, BUF - 1, 0, NULL, NULL);
        if (n < 0) {
            printf("No response from server (timeout)\n");
            continue;
```

```
        }
        reply[n] = '\0';
        printf("Server reply : %s\n", reply);
    }
    close(sock);
    printf("Client closed.\n");
    return 0;
}
```

### How to run

- **Linux / macOS:** `gcc dns_server.c -o dns_server` and `gcc dns_client.c -o dns_client`. Start the server in one terminal (`./dns_server 8053`) and the client in a second terminal (`./dns_client 127.0.0.1 8053`).
- **Windows (Code::Blocks / MinGW):** the programs already contain the Winsock code. Add the library **ws2_32** under *Settings → Compiler → Linker settings → Add* (or compile with `gcc dns_server.c -o dns_server -lws2_32`). Run dns_server.exe first, then dns_client.exe in a second window. Allow the program through Windows Firewall if asked.
- To use two machines, run the server on one PC and give its IP address to the client on the other.

### Output – client terminal

```
$ ./dns_client 127.0.0.1 8053
DNS client - server 127.0.0.1:8053

Enter host name or IP address (exit to quit): www.vit.ac.in
Server reply : FORWARD www.vit.ac.in -> 10.0.0.10 (authoritative, local table)

Enter host name or IP address (exit to quit): myweb.local
Server reply : FORWARD myweb.local -> 10.0.0.1 (authoritative, local table)

Enter host name or IP address (exit to quit): 10.0.0.20
Server reply : REVERSE 10.0.0.20 -> mail.vit.ac.in (authoritative, local table)

Enter host name or IP address (exit to quit): localhost
Server reply : FORWARD localhost -> 127.0.0.1 (authoritative, local table)

Enter host name or IP address (exit to quit): unknown.example
Server reply : FORWARD unknown.example -> NOT FOUND (NXDOMAIN)

Enter host name or IP address (exit to quit): 10.9.9.9
Server reply : REVERSE 10.9.9.9 -> NOT FOUND (NXDOMAIN)

Enter host name or IP address (exit to quit): exit
Client closed.
```

### Output – server terminal

```
$ ./dns_server 8053
DNS server running on UDP port 8053 with 6 records
Query from 127.0.0.1:58058  "www.vit.ac.in"  =>  FORWARD www.vit.ac.in -> 10.0.0.10 (authoritative,
    local table)
Query from 127.0.0.1:58058  "myweb.local"  =>  FORWARD myweb.local -> 10.0.0.1 (authoritative, local
    table)
Query from 127.0.0.1:58058  "10.0.0.20"  =>  REVERSE 10.0.0.20 -> mail.vit.ac.in (authoritative, local
    table)
Query from 127.0.0.1:58058  "localhost"  =>  FORWARD localhost -> 127.0.0.1 (authoritative, local table)
Query from 127.0.0.1:58058  "unknown.example"  =>  FORWARD unknown.example -> NOT FOUND (NXDOMAIN)
Query from 127.0.0.1:58058  "10.9.9.9"  =>  REVERSE 10.9.9.9 -> NOT FOUND (NXDOMAIN)
```

**Tested on:** The programs above were compiled and run on Linux to produce this output. The Windows-specific lines (inside `#ifdef _WIN32`) follow the standard Winsock pattern but were not compiled here, so if Code::Blocks reports an error, check that ws2_32 is linked.

*Output screenshot – Experiment 15 Part A (server and client windows)*

(paste your own screenshot here, with your ID card visible)

## Part B – DNS client-server in Cisco Packet Tracer

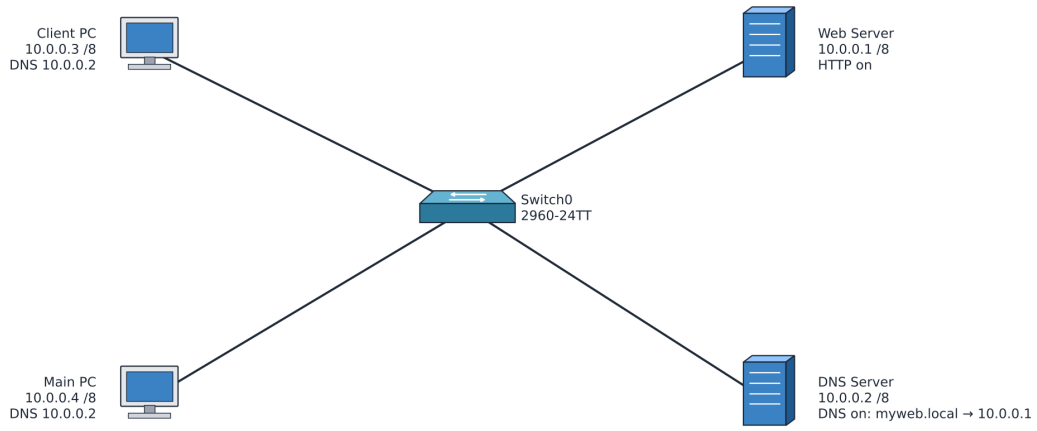



*Figure 15.3 – DNS client-server network in Packet Tracer*

### Step 1. Place the devices

Open Cisco Packet Tracer → *File* → *New*. From the device-type selection box at the bottom-left drag the following onto the Logical workspace (see the "Packet Tracer basics" section, Figure B.1):

- 1 × 2960 switch, 2 × Servers (Web Server, DNS Server), 2 × PCs (Client PC, Main PC)

Click each device and change its **Display Name** (Config tab → Global → Settings) to match the diagram, so the labels in your screenshots match the procedure.

### Step 2. Connect the cables

Click the **Connections** (lightning-bolt) icon. Choose the cable type, click the first device, pick the port from the pop-up list, then click the second device and pick its port:

- Copper straight-through from each device's FastEthernet0 to a switch port

Wait until the link lights turn **green** (switch ports take about 30 s to move from amber to green). Router ports stay **red** until you enter `no shutdown`.

### Step 3. Assign static IP addresses

| Device | IP address | Subnet mask | DNS server |
|---|---|---|---|
| Web Server | `10.0.0.1` | `255.0.0.0` | `10.0.0.2` |
| DNS Server | `10.0.0.2` | `255.0.0.0` | `10.0.0.2` |
| Client PC | `10.0.0.3` | `255.0.0.0` | `10.0.0.2` |
| Main PC | `10.0.0.4` | `255.0.0.0` | `10.0.0.2` |

### Step 4. Enable the services

Web Server → Services → **HTTP**: On (keep the default index.html). DNS Server → Services → **DNS**: On; add Name `myweb.local`, Type A Record, Address `10.0.0.1` → Add (Figure 10.2). Optionally add a second record such as `dns.local → 10.0.0.2`.

### Step 5. Resolve the host name

Client PC → Command Prompt:

```
C:\> nslookup myweb.local

Server: [10.0.0.2]
Address:  10.0.0.2

Non-authoritative answer:
Name:   myweb.local
Address:  10.0.0.1

C:\> ping myweb.local
Pinging 10.0.0.1 with 32 bytes of data:
Reply from 10.0.0.1: bytes=32 time<1ms TTL=128
```

### Step 6. Open the web page by name

Client PC → Desktop → **Web Browser** → URL `http://myweb.local` → Go. The “Cisco Packet Tracer – Welcome …” page from the Web Server is displayed, proving the name was resolved to 10.0.0.1.

### Step 7. Observe the DNS exchange

Simulation mode → filters DNS and HTTP. Repeat Step 6 and press Capture/Forward: first a DNS query goes Client PC → Switch → DNS Server and the DNS reply returns; only then does the HTTP (TCP) traffic go to the Web Server. Also try `nslookup unknown.local` – the server answers that the name does not exist.

*Screenshot – nslookup myweb.local and the web page opened by name*

(paste your own screenshot here, with your ID card visible)

## Result

The C DNS server answered forward queries (www.vit.ac.in → 10.0.0.10, myweb.local → 10.0.0.1) and reverse queries (10.0.0.20 → mail.vit.ac.in) from its table and returned NXDOMAIN for unknown names and addresses; the client displayed every answer. In Packet Tracer, the DNS server resolved myweb.local to 10.0.0.1, so the client could ping and browse the web server by name.

## Conclusion

A DNS client-server pair was built with UDP sockets, showing that name resolution is a simple request/response exchange over a connectionless transport, supporting both name-to-address and address-to-name lookups. The Packet Tracer setup demonstrated the same mechanism with a real DNS service.